

MÉMOIRE DU PROJET DE FIN D'ÉTUDE

PRÉSENTÉ EN VUE DE L'OBTENTION DU

Diplôme National d'Ingénieur d'État en Ingénierie des Données et Intelligence Artificielle

PAR

Sami TALEB

Modélisation des Données et Architecture d'Intelligence Artificielle pour une Plateforme de Réseautage Professionnelle

RÉALISÉ À :



SOUS LA DIRECTION DE :

M. Mourad EL YADARI,

ENSAM Rabat

Encadrant

M. Omar HAMRI,

HamriCapital

Encadrant

M. Hicham,

HamriCapital

Encadrant

SOUTENU LE [Date] JUIN 2026, DEVANT LE JURY COMPOSÉ DE :

Pr. Mourad EL YADARI,

ENSAM Rabat

Pr. [Nom de l'Examinateur],

ENSAM Rabat

Pr. [Nom de l'Examinateur],

ENSAM Rabat

Année universitaire 2025/2026

Dédicaces

À la mémoire de mon père, que Dieu l'accueille en Sa miséricorde.

J'aurais tellement aimé qu'il soit là aujourd'hui.

*À ma mère, qui a tout porté sur ses épaules pour que je puisse arriver
jusqu'ici.*

Ce diplôme est le tien avant d'être le mien.

*À mes frères et ma sœur, pour votre présence et votre patience durant ces
cinq années.*

*À mes amis, ceux avec qui j'ai partagé les nuits blanches, les deadlines et
les fous rires.*

Ce parcours n'aurait pas eu la même saveur sans vous.

Remerciements

Alhamdulillah, Le Très-Savant, Le Conquérant, le guide qui m'a guidé tout au long de ma vie et qui m'a donné la force d'accomplir mes études. Alhamdulillah pour mes parents, sources de motivation, d'espoir et de soutien, tant dans les moments de difficulté que dans les périodes de prospérité. Merci à Dieu pour tout.

Je remercie chaleureusement l'équipe de HamriCapital, et plus particulièrement mon encadrant industriel Monsieur Omar HAMRI, pour leur accueil et leur expertise, qui ont grandement contribué au succès de mon PFE.

Je tiens également à exprimer ma profonde gratitude à mon encadrant académique, Monsieur Mourad EL YADARI, pour sa guidance et ses conseils avisés durant ma période de stage, ainsi que pour ses efforts constants en faveur de la réussite de la formation INDIA.

Enfin, je remercie sincèrement les membres du jury pour le temps consacré à la lecture de mon rapport et pour l'honneur qu'ils me font par leur présence à ma soutenance.

Résumé

Ce rapport présente le travail que j'ai réalisé lors de mon projet de fin d'études au sein de l'entreprise HamriCapital. Le projet porte sur la conception et le développement de « The Communium », une plateforme de réseautage professionnel et une marketplace dédiée à l'écosystème entrepreneurial marocain. Pour répondre aux défis de la mise en relation ciblée et de la sécurité des échanges, une architecture logicielle découplée a été mise en place, intégrant de puissants modèles d'Intelligence Artificielle. Le travail s'est concentré sur la modélisation des données avec PostgreSQL et l'extension *pgvector* pour la recherche sémantique, ainsi que sur le développement d'algorithmes de traitement du langage naturel (NLP) pour l'appariement précis des profils et de vision par ordinateur (OCR trilingue) pour l'automatisation du processus de vérification d'identité (KYC). Ce projet m'a permis de déployer une infrastructure robuste et de démontrer la valeur ajoutée de l'IA dans l'optimisation des services professionnels.

Mots-clés : Plateforme professionnelle, Intelligence Artificielle, Recherche sémantique, *pgvector*, NLP, OCR, Microservices, KYC.

Abstract

This report presents the work I carried out during my end-of-studies project at HamriCapital. The project focuses on the design and development of “The Communion”, a professional networking platform and marketplace dedicated to the Moroccan entrepreneurial ecosystem. To address the challenges of targeted matchmaking and secure exchanges, a decoupled software architecture was implemented, integrating powerful Artificial Intelligence models. The work focused on data modeling using PostgreSQL and the *pgvector* extension for semantic search, as well as the development of Natural Language Processing (NLP) algorithms for accurate profile matching and computer vision (trilingual OCR) to automate the identity verification (KYC) process. This project allowed me to deploy a robust infrastructure and demonstrate the added value of AI in optimizing professional services.

Keywords : Professional platform, Artificial Intelligence, Semantic search, pgvector, NLP, OCR, Microservices, KYC.

Table des matières

Dédicaces	ii
Remerciements	iii
Résumé	iv
Abstract	v
Liste des Abréviations	x
Table des figures	xiii
Liste des tableaux	xiv
Introduction Générale	1
1 Cadrage du Projet	2
1.1 Présentation de l’Organisme d’Accueil	2
1.1.1 L’Organisme : HamriCapital	2
1.1.2 Positionnement Stratégique et Vision	2
1.1.3 Secteurs d’Intervention	3
1.1.4 La Division Technologie et Innovation Digitale	3
1.2 Cadre Général du Projet	4
1.2.1 Présentation de The Communium	4
1.2.2 Problématique	4
1.3 Analyse de l’Existant et Benchmark	6
1.3.1 Analyse Critique des Solutions Actuelles	6
1.3.2 Tableau Comparatif et Benchmark	7
1.3.3 Positionnement de The Communium	7
1.4 Définition des Objectifs et Périmètre du Projet	7
1.4.1 Objectifs Fonctionnels	7
1.4.2 Objectifs Techniques	8
1.4.3 Périmètre du Mémoire (Focus Data & Intelligence Artificielle)	8

1.5	Planification du Projet	9
1.5.1	Méthodologie Agile et Adaptation au Contexte	9
1.5.2	Backlog Produit (Focus Data & IA)	9
1.5.3	Phases du Projet et Diagramme de Gantt	10
	Conclusion	11
2	État de l'Art et Étude Théorique	12
2.1	Recherche Sémantique et Représentations Vectorielles	12
2.1.1	Le Problème de la Recherche par Mots-Clés	12
2.1.2	Du Mot au Vecteur : Évolution des Représentations	12
2.1.3	Sentence-Transformers : Des Embeddings au Niveau de la Phrase	13
2.1.4	Choix du Modèle : paraphrase-multilingual-MiniLM-L12-v2	14
2.1.5	Indexation Vectorielle : pgvector	14
2.2	Vision par Ordinateur et OCR	14
2.2.1	Prétraitement d'Image	15
2.2.2	Reconnaissance Optique de Caractères	15
2.3	Systèmes de Recommandation	16
2.3.1	Filtrage Collaboratif vs. Filtrage par Contenu	16
2.3.2	Maximal Marginal Relevance (MMR)	16
2.4	Détection d'Anomalies et Statistiques Robustes	16
2.4.1	Z-Score Classique et ses Limites	17
2.4.2	Z-Score Modifié par la MAD	17
2.4.3	Entropie de Shannon pour la Détection de Faux Avis	17
2.5	Prédiction d'Attrition Client (Churn)	18
2.5.1	Analyse RFM	18
2.5.2	Customer Lifetime Value (CLV)	18
2.6	Classification Automatique et Analyse de Sentiment	18
2.6.1	Classification Zero-Shot par Similarité Sémantique	19
2.6.2	Analyse de Sentiment des Avis	19
2.7	Estimation des Délais de Livraison	20
2.8	Benchmark des Technologies	21
	Conclusion	22
3	Conception et Architecture	23
3.1	Architecture Globale du Système	23

3.1.1	Organisation en Monorepo (PNPM Workspaces + Turborepo)	23
3.1.2	Architecture Micro-services en Trois Tiers	24
3.1.3	Justification de la Séparation NestJS / FastAPI	25
3.2	Pile Technologique	25
3.3	Architecture des Données	26
3.3.1	Schéma Prisma et Domaines Fonctionnels	26
3.3.2	Le Modèle Utilisateur, Point Central	27
3.3.3	Intégration de pgvector avec Prisma	27
3.3.4	Stratégie de Cache (Redis)	28
3.4	Modélisation Fonctionnelle (UML)	28
3.4.1	Acteurs du Système	28
3.4.2	Diagrammes de Cas d'Utilisation	28
3.4.3	Diagrammes de Séquence	30
3.4.4	Diagramme d'Activité de l'Onboarding	31
3.5	Architecture du Service d'Intelligence Artificielle	31
3.5.1	Génération d'Embeddings Sémantiques	32
3.5.2	Moteur de Similarité Vectorielle	32
3.5.3	Appariement Intelligent de Mentorat	32
3.5.4	Suggestion de Prix et Détection d'Anomalies	33
3.5.5	Classification Zero-Shot et Extraction de Tags	34
3.5.6	Analyse de Sentiment des Avis	34
3.5.7	Prédiction d'Attrition (Churn) et Calcul de CLV	35
3.5.8	Prédiction des Délais de Livraison (ETA)	35
3.5.9	Recommandation Diversifiée via MMR	35
3.5.10	Extraction de Données CNIE par OCR	35
3.6	Sécurité	36
	Conclusion	36
4	Réalisation et Analyse des Résultats	38
4.1	Environnement Technique	40
4.2	Workflow de Développement	40
4.3	Présentation des Interfaces	40
4.3.1	Onboarding et KYC / KYB	40
4.3.2	Fil d'Actualité et Réseau Social	40

4.3.3	Marketplace et Enchères	40
4.3.4	Espace Mentorat	40
4.3.5	Messagerie Instantanée	40
4.3.6	Tableau de Bord Administrateur	40
4.4	Démonstration des Workflows IA	40
4.4.1	Création d'Annonce Enrichie par l'IA	40
4.4.2	Vérification KYC Automatisée	40
4.5	Stratégie de Tests et Validation	40
4.5.1	Tests Unitaires IA (pytest)	40
4.5.2	Tests d'Intégration API	40
4.5.3	Validation des Données	40
4.6	Évaluation et Métriques	40
4.6.1	Métriques OCR	40
4.6.2	Métriques de Recommandation	40
4.6.3	Métriques de Détection (Anomalies, Fraude, Churn)	40
4.6.4	KPI Business et Impact	40
	Conclusion Générale et Perspectives	41
	A Structure du Monorepo	42
	B Schéma Prisma (Extraits Clés)	44
	C Endpoints du Microservice IA	47
	D Résultats des Tests Unitaires	49
D.1	Prédiction d'Attrition (Churn) et Réengagement	50
D.2	Extraction de Données via Vision par Ordinateur (OCR CNIE)	51
D.3	Classification Zero-Shot et Extraction de Tags (NLP)	52
D.4	Moteur de Recherche Sémantique : Génération d'Embeddings	53
D.5	Prédiction des Délais de Livraison (Modèle ETA)	54
D.6	Appariement Hybride des Mentors	55
D.7	Algorithme de Recommandations Diversifiées (MMR)	56
D.8	Modèle d'Estimation de Prix (Pricing)	57
D.9	Analyse de Sentiments Multilingue	58
D.10	Recherche de Similarité Vectorielle	59
	E Captures d'Écran de l'Application	60

Liste des Abréviations

IA	Intelligence Artificielle
NLP	Natural Language Processing
OCR	Optical Character Recognition
CNIE	Carte Nationale d'Identité Électronique
KYC	Know Your Customer
KYB	Know Your Business
MMR	Maximal Marginal Relevance
CLV	Customer Lifetime Value
RFM	Récence, Fréquence, Valeur Monétaire
MAD	Median Absolute Deviation
ORM	Object-Relational Mapping
API	Application Programming Interface
SSR	Server-Side Rendering
JWT	JSON Web Token
DTO	Data Transfer Object
CRUD	Create, Read, Update, Delete
HNSW	Hierarchical Navigable Small World
RBAC	Role-Based Access Control
CER	Character Error Rate
WER	Word Error Rate
ILD	Intra-List Diversity
NDCG	Normalized Discounted Cumulative Gain
UML	Unified Modeling Language

Table des figures

1.1	logo de l'entreprise.	2
1.2	Organigramme de HamriCapital et positionnement du PFE	3
1.3	Écosystème cible et interactions professionnelles à 360° au sein de The Communium	4
1.4	logo de LinkedIn.	6
1.5	Diagramme de Gantt du déroulement réel du projet	11
2.1	Architecture siamoise de Sentence-BERT (SBERT) pour la comparaison sémantique	13
3.1	Organisation du monorepo PNPM + Turborepo	24
3.2	Architecture micro-services en trois tiers de The Communium	24
3.3	Vue d'ensemble des domaines du schéma de données (Prisma)	27
3.4	Le modèle User et ses relations principales	27
3.5	Diagramme de cas d'utilisation — Module Marketplace	29
3.6	Diagramme de cas d'utilisation — Module Mentorat	29
3.7	Diagramme de cas d'utilisation — Module KYC / KYB	29
3.8	Diagramme de cas d'utilisation — Module IA	30
3.9	Diagramme de séquence — Création d'annonce enrichie par l'IA	30
3.10	Diagramme de séquence — Vérification KYC par OCR de la CNIE	31
3.11	Diagramme d'activité — Processus d'intégration (Onboarding)	31
3.12	Architecture interne du microservice d'Intelligence Artificielle	32
3.13	Pipeline de génération d'un embedding	32
3.14	Calcul du score hybride d'appariement de mentorat	33
A.1	Architecture globale du monorepo (Turborepo/pnpm) séparant les applications (apps) et les modules partagés (packages).	42
A.2	Arborescence principale de l'application ai-service, montrant la configuration Docker, les tests et le module central app.	43
A.3	Détail du dossier routers définissant les points de terminaison de l'API.	43
A.4	Détail du dossier services contenant les algorithmes d'IA et de Machine Learning. .	43

B.1	Structure du paquet partagé database, contenant le schéma Prisma, les migrations et le client ORM utilisé par l'ensemble du projet.	44
C.1	En-tête de l'API et points de terminaison pour la génération d'embeddings multilingues et le calcul de similarité vectorielle.	47
C.2	Points de terminaison dédiés à l'appariement hybride des mentors, la suggestion de prix, la classification NLP <i>Zero-Shot</i> et l'analyse de sentiment.	47
C.3	Points de terminaison pour la prédiction d'attrition (Churn RFM), les recommandations diversifiées (MMR), l'estimation des délais (ETA) et l'extraction de données par vision par ordinateur (OCR CNIE).	48
C.4	Route de vérification de l'état du serveur (Health Check) et aperçu des schémas de données de validation générés via Pydantic.	48
D.1	Requête JSON contenant les données d'activité de deux utilisateurs distincts.	50
D.2	Génération dynamique d'actions de réengagement.	50
D.3	Interface de test Swagger pour le téléversement sécurisé (multipart/form-data) d'une image de CNIE.	51
D.4	Résultat de l'extraction OCR : identité complète, numéro de carte et date de naissance.	51
D.5	Soumission d'une annonce brute pour analyse sémantique.	52
D.6	Réponse du service : catégorie technique, niveau d'urgence et liste des tags pertinents.	52
D.7	Requête envoyant le titre et la description d'une annonce (ex. : MacBook Pro).	53
D.8	Vecteur multidimensionnel (embedding) généré avec succès par le modèle NLP.	53
D.9	Exemple de requête paramétrée pour le calcul du délai d'expédition.	54
D.10	Estimation du délai, prenant en compte la distance géographique (trajet interurbain).	54
D.11	Requête envoyant le vecteur d'intérêts de l'utilisateur et les données des mentors candidats.	55
D.12	Classement final appliquant les pondérations métier et les filtres de disponibilité.	55
D.13	Configuration du point de terminaison, incluant le paramètre de diversité lambda.	56
D.14	Résultat MMR : tri final équilibrant le score de pertinence et la pénalité de redondance.	56
D.15	Requête soumettant les caractéristiques d'un ordinateur portable d'occasion (24 mois, bon état).	57
D.16	Déclenchement de la méthode heuristique de secours (<i>heuristic_cold</i>) calculant la décote du matériel.	57
D.17	Requête soumettant un lot de textes hétérogènes (anglais, français, arabe) pour analyse de polarité.	58

D.18 Prédiction du modèle NLP : polarité (POSITIVE/NEGATIVE) et score de confiance par langue.	58
D.19 Requête soumettant l'embedding d'une annonce cible et ceux de trois annonces candidates.	59
D.20 Similarité cosinus : les deux annonces les plus pertinentes (top_k=2), triées par proximité.	59
E.1 Page d'accueil (Landing Page) de The Communium	60
E.2 Processus d'inscription et de création de compte	60
E.3 Processus de vérification d'identité (KYC)	61
E.4 Finalisation de l'intégration (Onboarding)	61
E.5 Fil d'actualité et suggestions de connexions	62
E.6 Tableau de bord (Dashboard) résumant l'activité de l'utilisateur	62
E.7 Vue détaillée du profil utilisateur et de sa fiche d'identité vérifiée	63
E.8 Menus de navigation principaux de la plateforme	63
E.9 Interface de l'espace Forum catégorisé	64
E.10 Marketplace B2B/B2C pour l'achat et la vente de services ou produits	64
E.11 Page de gestion des paramètres et de la sécurité du compte	65

Liste des tableaux

1.1	Tableau comparatif des fonctionnalités face à la concurrence	7
2.1	Benchmark des technologies évaluées pour chaque couche	21
3.1	Pile technologique de The Communium par couche	26
3.2	Multiplicateurs de prix selon l'état du bien	34

Introduction Générale

L'intelligence artificielle redessine aujourd'hui la manière dont les plateformes numériques fonctionnent, de la personnalisation des contenus à la détection de fraudes. Des secteurs comme le commerce en ligne, la finance ou les réseaux professionnels s'appuient de plus en plus sur des algorithmes d'apprentissage automatique et de traitement du langage naturel pour offrir des expériences plus pertinentes à leurs utilisateurs.

Au Maroc, l'écosystème entrepreneurial est en pleine expansion, mais les professionnels évoluent dans un paysage numérique fragmenté : LinkedIn est pensé pour un marché anglo-saxon, les plateformes de petites annonces ne proposent pas de mise en relation professionnelle, et aucune solution existante ne combine réseau social, marketplace et services intelligents dans un environnement trilingue adapté au contexte local.

C'est dans ce contexte que s'inscrit mon projet de fin d'études, réalisé au sein de **HamriCapital**, une société d'ingénierie financière basée à Tanger. La problématique qui guide ce travail peut se formuler ainsi : *comment concevoir une architecture de données et d'intelligence artificielle capable d'offrir des services personnalisés — recherche sémantique, vérification d'identité automatisée, recommandation diversifiée, détection de fraude — au sein d'une plateforme professionnelle adaptée au marché marocain ?*

Mon travail s'est articulé autour de deux piliers : la modélisation des données à l'aide de PostgreSQL et de l'extension pgvector pour la recherche sémantique, ainsi que le développement d'un microservice IA intégrant des embeddings multilingues, un pipeline d'OCR pour la carte d'identité nationale, un algorithme de recommandation diversifiée, un système de mentorat intelligent, et des modules de détection d'anomalies et de fraude.

Ce rapport est structuré en quatre chapitres. Le premier présente le cadrage du projet : organisme d'accueil, analyse de l'existant et planification. Le deuxième dresse un état de l'art des domaines concernés et justifie les choix technologiques. Le troisième détaille la conception et l'architecture de la solution, depuis la modélisation des données jusqu'à la formalisation de chaque algorithme. Enfin, le quatrième expose la réalisation, les tests et l'évaluation des résultats obtenus.

Chapitre 1

Cadrage du Projet

Introduction

Ce premier chapitre a pour objectif de poser les fondations du projet de fin d'études. Il débute par une présentation de l'organisme d'accueil, HamriCapital, avant de détailler le cadre général du projet *The Communium*. Nous y analyserons les problématiques de l'écosystème entrepreneurial marocain actuel, dresserons un bilan des solutions existantes, et définirons les objectifs précis ainsi que la planification adoptée pour la réalisation de ce projet.

1.1 Présentation de l'Organisme d'Accueil

1.1.1 L'Organisme : HamriCapital



FIGURE 1.1 – logo de l'entreprise.

Fondée au premier trimestre 2025, HamriCapital est une société de gestion d'investissement internationale dont le siège social est établi à Tanger, au Maroc. Portée par une ambition mondiale dès sa création, l'entreprise s'est étendue en ouvrant des bureaux satellites au Canada, plus précisément à Vancouver et à Toronto. Dirigée par son CEO, M. Omar Hamri, la société gère un portefeuille en forte croissance de plus de 20 millions de dollars, réparti sur plus de 25 investissements actifs et structuré autour de 8 stratégies multi-classes d'actifs distinctes.

1.1.2 Positionnement Stratégique et Vision

HamriCapital se positionne de manière unique à l'intersection de la finance institutionnelle et de l'entrepreneuriat technologique. Sa mission principale est de servir de pont stratégique entre les marchés émergents, notamment le Maroc et l'Afrique, et les marchés développés nord-américains.

et européens. La gouvernance de l'entreprise s'appuie sur quatre piliers fondamentaux : le devoir fiduciaire (garantissant que les intérêts du client priment toujours), l'excellence de la performance via une recherche systématique, une perspective mondiale, et une approche partenariale axée sur la confiance et la transparence.

1.1.3 Secteurs d'Intervention

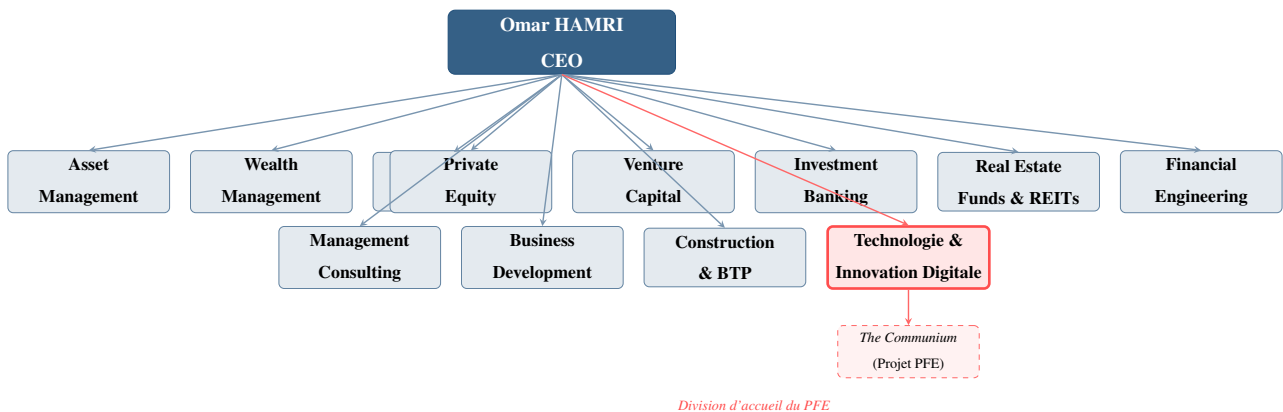


FIGURE 1.2 – Organigramme de HamriCapital et positionnement du PFE

Afin de construire un portefeuille résilient, la stratégie d'investissement de HamriCapital couvre un large spectre sectoriel. La *Technologie et l'Innovation Digitale* y occupe une place centrale, englobant des domaines de pointe tels que l'intelligence artificielle, la cybersécurité, le cloud souverain et les agents autonomes. La société investit également massivement dans les services financiers (FinTech), la santé (biotech et recherche médicale), les énergies renouvelables, l'immobilier, ainsi que la logistique et l'industrie.

1.1.4 La Division Technologie et Innovation Digitale

C'est au sein de la division *Technologie et Innovation Digitale* que s'est déroulé ce projet de fin d'études. Cette division est spécifiquement chargée de piloter les projets numériques internes de la société et de concevoir des solutions agissant comme de véritables vitrines technologiques de l'expertise du groupe. L'environnement de travail, semblable à celui d'une startup ambitieuse, favorise l'autonomie, la prise de décision rapide et l'innovation continue. C'est dans ce cadre stimulant, et sous la supervision directe de la direction, qu'a émergé le développement de *The Communium*, conçu pour fédérer l'écosystème de partenaires, d'investisseurs et de collaborateurs de HamriCapital.

1.2 Cadre Général du Projet

1.2.1 Présentation de The Communium

The Communium est un réseau social et une place de marché professionnelle (marketplace) à destination des entrepreneurs, freelances et entreprises, pensé spécifiquement pour le marché marocain. Né de la volonté de combler le vide laissé par les plateformes internationales souvent inadaptées aux spécificités locales (culturelles, linguistiques et réglementaires), ce projet vise à réunir sur un espace numérique unique tout ce dont un professionnel a besoin pour développer son réseau et ses affaires.

La plateforme propose une expérience à 360 degrés intégrant la gestion des profils professionnels, la messagerie en temps réel, l'organisation d'événements, le mentorat et les transactions commerciales. Afin de répondre aux exigences de son audience, elle est conçue nativement de manière trilingue (français, anglais, arabe). Dans le cadre de ce projet de fin d'études mené en totale autonomie, mon travail se concentre spécifiquement sur le cœur névralgique de cette plateforme : la modélisation d'une architecture de données robuste et l'intégration de services d'Intelligence Artificielle avancés.

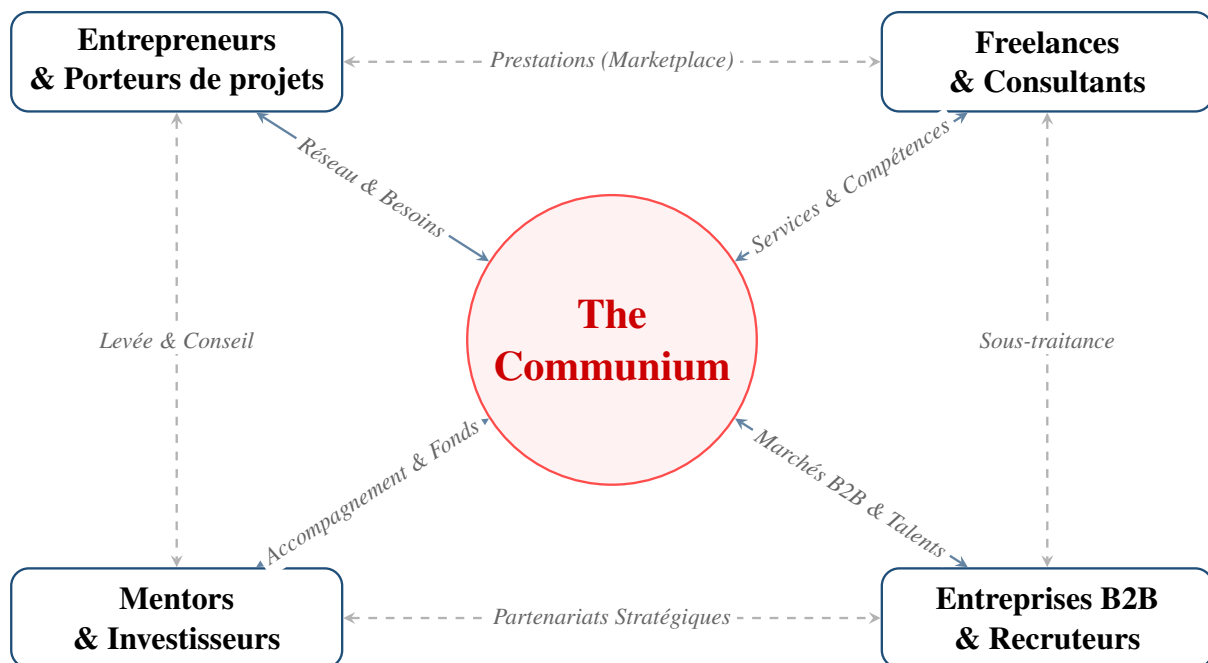


FIGURE 1.3 – Écosystème cible et interactions professionnelles à 360° au sein de The Communium

1.2.2 Problématique

Le développement de *The Communium* répond à des défis majeurs identifiés au sein de l'écosystème entrepreneurial et des solutions numériques actuelles :

- **Fragmentation et inadaptation des outils :** Les professionnels marocains manquent d'un espace numérique centralisé et véritablement adapté à leur réalité. Il est difficile de fédérer des partenaires, des clients et des collaborateurs au sein d'un écosystème maîtrisé et sécurisé.
- **Lourdeur des vérifications d'identité (KYC/KYB) :** Dans les environnements professionnels, la confiance est primordiale. Or, la vérification manuelle des pièces d'identité (CNIE) ou des documents d'entreprise est un processus chronophage qui crée une forte friction lors de l'intégration de nouveaux membres.
- **Redondance des recommandations :** La plupart des systèmes de recommandation s'appuient sur des tris basiques de similarité, ce qui produit un effet de redondance (la « bulle de filtres ») empêchant les utilisateurs de découvrir des annonces ou des profils variés et pertinents.
- **Vulnérabilité face à la fraude :** Les places de marché souffrent fréquemment d'anomalies de prix et de campagnes de faux avis, nuisant gravement à la crédibilité de la plateforme et à la sécurité des utilisateurs.

La problématique centrale de mon travail s’articule donc autour de la question suivante : *Comment concevoir une architecture de données performante et déployer des modèles d’intelligence artificielle (vision par ordinateur, traitement du langage naturel, statistiques robustes) capables d’automatiser le KYC, de diversifier les recommandations et de détecter la fraude au sein d’une plateforme professionnelle à grande échelle ?*

1.3 Analyse de l’Existant et Benchmark

L’évaluation du paysage numérique actuel montre que les professionnels marocains doivent fragmenter leurs activités entre plusieurs outils, dont aucun ne répond parfaitement aux réalités socio-économiques et linguistiques du marché local. Cette section dresse un état des lieux des forces et des faiblesses des principaux acteurs existants afin de positionner précisément *The Communium*.

1.3.1 Analyse Critique des Solutions Actuelles

- **LinkedIn** : C’est la référence mondiale pour le réseautage professionnel. Cependant, son modèle est pensé pour les marchés occidentaux et anglo-saxons. La plateforme ignore les spécificités réglementaires locales (comme l’authentification officielle des entreprises marocaines) et n’offre aucune fonctionnalité de commerce intégré (marketplace B2B/B2C) ou de gestion de transactions financières en Dirhams (MAD).



FIGURE 1.4 – logo de LinkedIn.

- **Avito.ma** : Acteur historique des petites annonces au Maroc, il excelle dans le volume de transactions grand public. Néanmoins, il souffre d’un manque total de cadre professionnel : l’identité des vendeurs n’est pas vérifiée, la fraude et les prix aberrants y sont fréquents, et il est impossible d’y bâtir un réseau de partenaires ou de mentors.

1.3.2 Tableau Comparatif et Benchmark

Le tableau 1.1 synthétise les fonctionnalités clés de *The Communium* face aux solutions dominantes du marché.

TABLE 1.1 – Tableau comparatif des fonctionnalités face à la concurrence

Fonctionnalités / Critères	LinkedIn	Avito	The Communium
Réseautage professionnel & Profils	Très fort	Aucun	Fort (Vérifié)
Marketplace intégrée	Aucune	Forte (B2C)	Forte (B2B/B2C)
Support trilingue (FR / AR / EN)	Partiel	Aucun	Natif & Complet
Vérification d'identité (KYC/KYB)	Limite	Aucun	Automatisé (IA)
Messagerie & Événements Temps Réel	Oui	Limité	Oui (Socket.io)
Économie de Tokens & Paiements MA	Aucun	Aucun	Intégré (Stripe/CMI)
Détection anti-fraude algorithmique	Modérée	Aucune	Avancée (Robust Stat)

1.3.3 Positionnement de The Communium

Comme le démontre cette analyse, *The Communium* se positionne comme la première plateforme tiers unifiée sur le marché marocain. En fusionnant les dynamiques relationnelles d'un réseau social professionnel avec l'efficacité transactionnelle d'une marketplace sécurisée, elle comble un vide stratégique. L'apport majeur de l'application réside dans l'intégration de sa couche d'Intelligence Artificielle et de modélisation de données (sujet central de ce mémoire) : elle transforme un espace d'échange classique en un écosystème intelligent, automatisé et protégé, capable de soutenir de bout en bout l'activité des entrepreneurs et freelances nationaux.

1.4 Définition des Objectifs et Périmètre du Projet

Pour répondre à la problématique identifiée, le projet *The Communium* s'articule autour d'objectifs fonctionnels et techniques précis, tout en délimitant un périmètre académique clair pour ce mémoire de fin d'études.

1.4.1 Objectifs Fonctionnels

L'application a été conçue pour offrir une valeur ajoutée immédiate à ses utilisateurs à travers les objectifs suivants :

- **Fédérer l'écosystème** : Créer un réseau unifié permettant aux entrepreneurs, freelances et entreprises marocaines d'interagir, de publier des offres et de trouver des opportunités d'affaires.
- **Fluidifier l'intégration (Onboarding)** : Automatiser l'extraction et la vérification des données des pièces d'identité (KYC) pour réduire le temps d'inscription de plusieurs minutes à quelques secondes, sans intervention humaine.
- **Personnaliser l'expérience** : Fournir un moteur de recommandation intelligent pour les annonces de la marketplace et l'appariement avec des mentors, en évitant l'effet de redondance.
- **Garantir la sécurité et la confiance** : Détecter de manière proactive les comportements frauduleux, tels que les prix aberrants ou les campagnes de faux avis.

1.4.2 Objectifs Techniques

Sur le plan de l'ingénierie logicielle et de la donnée, le projet vise à :

- **Déployer une architecture moderne et découplée** : Séparer la logique transactionnelle classique des calculs algorithmiques lourds via une approche micro-services.
- **Implémenter la recherche vectorielle** : Dépassez la recherche textuelle basique en intégrant des bases de données capables de comprendre le sens sémantique des requêtes (embeddings).
- **Intégrer des modèles de Machine Learning en production** : Mettre en place des pipelines d'Intelligence Artificielle (Computer Vision, NLP) optimisés pour des temps de réponse faibles.

1.4.3 Périmètre du Mémoire (Focus Data & Intelligence Artificielle)

Bien que la réalisation de *The Communium* ait nécessité un développement *Full-Stack* complet mené en autonomie (incluant le Frontend en Next.js, la gestion des paiements, et la messagerie en temps réel), **le présent mémoire se concentre exclusivement sur les couches Data et Intelligence Artificielle**, conformément aux exigences de la filière Ingénierie des Données et Intelligence Artificielle (INDIA).

Ainsi, les chapitres suivants détailleront spécifiquement :

- La modélisation de la base de données PostgreSQL et l'utilisation avancée de l'extension *pgvector*.
- L'architecture du microservice IA développé en Python (FastAPI).
- Les pipelines d'extraction sémantique (OCR trilingue pour la CNIE marocaine).
- Les algorithmes décisionnels mis en œuvre (Maximal Marginal Relevance pour la diversité, Z-Score modifié pour la fraude, etc.).

1.5 Planification du Projet

Face à l'envergure de *The Communium* et à la complexité des modules d'Intelligence Artificielle à intégrer, une gestion de projet rigoureuse était indispensable. L'objectif était de maintenir un rythme de livraison constant tout en gardant la flexibilité nécessaire pour ajuster les modèles algorithmiques en fonction des premiers résultats.

1.5.1 Méthodologie Agile et Adaptation au Contexte

Pour mener à bien ce développement, j'ai opté pour une approche itérative et incrémentale inspirée du framework **Scrum**. Étant donné qu'il s'agit d'un projet de fin d'études mené en totale autonomie sur le plan technique, les rôles et cérémonies classiques du Scrum ont été adaptés à cette réalité :

- **Les Rôles** : J'ai assumé le rôle de l'équipe de développement (conception Data/IA, backend, frontend). Le rôle de *Product Owner* a été naturellement endossé par mon encadrant industriel, M. Omar Hamri, qui définissait les priorités métiers de HamriCapital et validait les livrables.
- **Les Sprints** : Le travail a été découpé en itérations courtes (sprints) de deux semaines. Chaque sprint se terminait par une démonstration fonctionnelle à la direction, permettant de valider les modèles de Machine Learning ou les nouvelles structures de base de données.
- **Les Cérémonies** : Les *Daily Stand-ups* ont été remplacés par un suivi personnel quotidien via un tableau Kanban (Trello/GitHub Projects), complété par une réunion de synchronisation hebdomadaire avec mes encadrants pour lever les points de blocage.

1.5.2 Backlog Produit (Focus Data & IA)

Le *Product Backlog* regroupait l'ensemble des fonctionnalités de la plateforme. Toutefois, dans le cadre restreint de ce mémoire axé sur l'Ingénierie des Données et l'IA, les *User Stories* (US) critiques ayant guidé mes développements étaient les suivantes :

- *US-01 (KYC Automatisé)* : « En tant que nouvel utilisateur, je veux importer une photo de ma carte d'identité afin que le système extraie automatiquement mes données (OCR) et valide mon profil. »
- *US-02 (Recherche Sémantique)* : « En tant qu'utilisateur, je veux rechercher des mentors ou des annonces en utilisant des phrases naturelles, afin d'obtenir des résultats basés sur le sens et non sur des mots-clés exacts. »
- *US-03 (Diversité MMR)* : « En tant qu'utilisateur naviguant sur la marketplace, je veux voir des recommandations pertinentes mais diversifiées, pour ne pas être inondé par des offres identiques.

»

- *US-04 (Anti-fraude)* : « En tant qu'avant-garde, je veux que le système détecte et me signale automatiquement les prix aberrants ou les faux avis générés par des bots. »

1.5.3 Phases du Projet et Diagramme de Gantt

Le développement de la plateforme *The Communium* s'est étalé sur cinq mois (février à juin 2026), structuré en six phases suivant une approche itérative inspirée de Scrum :

1. **Phase 1 – Cadrage et Architecture (Février)** : Analyse des besoins fonctionnels et non-fonctionnels, conception de l'architecture micro-services en trois tiers (Next.js / NestJS / FastAPI) organisée en monorepo PNPM + Turborepo, et modélisation du schéma Prisma (PostgreSQL 16 avec extension *pgvector*).
2. **Phase 2 – Backend et API (Mars)** : Mise en place du socle NestJS 10, intégration de Prisma 5 et du cache Redis, exposition des endpoints REST/GraphQL documentés via Swagger, et intégration des services tiers (Stripe/CMI, Cloudflare R2, Socket.IO).
3. **Phase 3 – Frontend (Avril – Début Mai)** : Développement des interfaces sous Next.js 14 (TailwindCSS, shadcn/ui) avec support trilingue (fr/ar/en), intégration de l'authentification NextAuth/JWT et du KYC, puis connexion REST et WebSockets au backend.
4. **Phase 4 – IA et NLP (Mai)** : Déploiement du microservice FastAPI, implémentation du matching sémantique (embeddings MiniLM-L12-v2 + MMR), du pipeline OCR de la CNIE (EasyOCR + OpenCV), des modules de détection d'anomalies (Z-Score MAD, entropie de Shannon) et de prédiction du churn/CLV.
5. **Phase 5 – Tests, DevOps et Déploiement (Fin Mai – Début Juin)** : Conteneurisation Docker Compose de l'ensemble des services, tests d'intégration globaux et tests unitaires des modules IA (CER, NDCG, F1), suivis des corrections et optimisations (cache Redis, indexation Meilisearch).
6. **Phase 6 – Clôture du projet (Juin)** : Rédaction du mémoire, préparation des supports de soutenance et présentation finale devant le jury.

Le diagramme de Gantt ci-dessous illustre visuellement cette répartition temporelle :

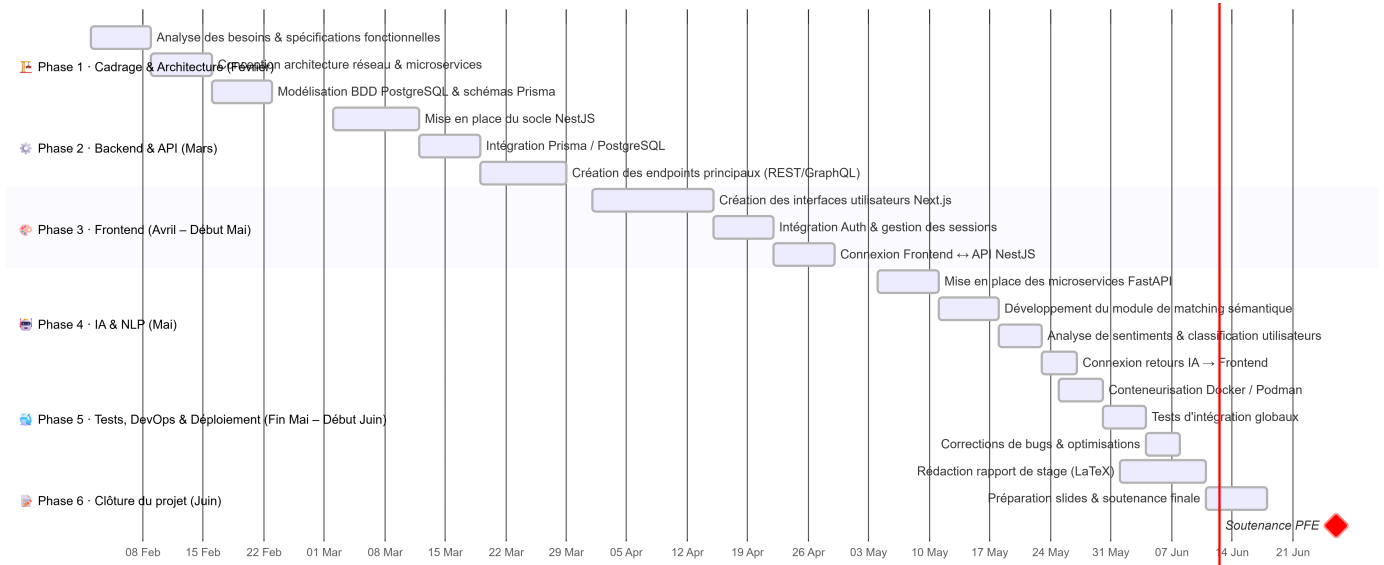


FIGURE 1.5 – Diagramme de Gantt du déroulement réel du projet

Conclusion

Ce premier chapitre a permis de définir les contours du projet *The Communium*. En partant des besoins de HamriCapital et des lacunes du marché marocain, nous avons pu formuler une problématique claire centrée sur l'apport de la donnée et de l'intelligence artificielle pour sécuriser et personnaliser le réseautage professionnel. La méthodologie Agile adoptée et la planification rigoureuse ont posé un cadre de travail solide pour les mois de développement. Le chapitre suivant s'attachera à explorer l'état de l'art des technologies sous-jacentes à notre solution, afin de justifier les choix algorithmiques et architecturaux qui guideront notre conception.

Chapitre 2

État de l'Art et Étude Théorique

Introduction

Avant de se lancer dans la conception technique, il est indispensable de comprendre les briques théoriques qui sous-tendent chaque composant intelligent de la plateforme. Ce chapitre passe en revue les travaux existants dans les domaines que touche *The Communium* : la recherche sémantique par embeddings, la vision par ordinateur appliquée à la lecture de documents d'identité, les systèmes de recommandation, les méthodes statistiques de détection d'anomalies et de fraude, et la modélisation du comportement utilisateur. Chaque section se termine par la justification du choix retenu pour notre projet. Le chapitre se clôt par un benchmark technologique comparant les principaux frameworks et outils disponibles.

2.1 Recherche Sémantique et Représentations Vectorielles

2.1.1 Le Problème de la Recherche par Mots-Clés

Lorsqu'un utilisateur tape « consultant marketing digital » dans une barre de recherche classique, le système va chercher ces mots exacts dans la base de données. Si un mentor s'est décrit comme « expert en stratégie digitale et communication en ligne », il ne sera tout simplement pas trouvé — alors qu'il correspond parfaitement à la demande. C'est la limite fondamentale de la recherche lexicale : elle ne comprend pas le *sens* des mots, seulement leur forme.

Pour dépasser cette limitation, la communauté du traitement du langage naturel (NLP) a développé des méthodes permettant de transformer du texte en vecteurs numériques — des *embeddings* — de sorte que deux phrases ayant un sens proche se retrouvent proches dans l'espace vectoriel, même si elles ne partagent aucun mot.

2.1.2 Du Mot au Vecteur : Évolution des Représentations

L'idée de représenter les mots par des vecteurs n'est pas nouvelle. Les premiers travaux marquants remontent à **Word2Vec mikolov2013**, proposé par Mikolov et al. en 2013, qui entraîne des vecteurs de mots sur de larges corpus textuels en se basant sur le contexte d'apparition de chaque mot. L'hypothèse

distributionnelle — « un mot se définit par la compagnie qu’il tient » — permet d’obtenir des vecteurs où les mots sémantiquement proches se retrouvent voisins. **GloVe pennington2014**, développé à Stanford, adopte une approche similaire mais s’appuie sur une matrice de co-occurrence globale plutôt que sur des fenêtres de contexte locales.

Ces modèles ont un défaut majeur : chaque mot possède un vecteur unique, quelle que soit la phrase. Le mot « banque » aura la même représentation qu’il désigne un établissement financier ou le bord d’une rivière. C’est l’arrivée des **Transformers vaswani2017** en 2017, avec leur mécanisme d’attention, qui a changé la donne. Le modèle **BERT devlin2019**, proposé par Google en 2018, est le premier à produire des représentations contextuelles : le vecteur d’un mot dépend de toute la phrase qui l’entoure.

2.1.3 Sentence-Transformers : Des Embeddings au Niveau de la Phrase

BERT produit des représentations riches, mais il n’est pas directement adapté à la comparaison de phrases. Comparer deux phrases avec BERT nécessite de les passer ensemble dans le modèle, ce qui rend la recherche sur des milliers de documents extrêmement lente (complexité quadratique).

Sentence-BERT (SBERT) reimers2019, proposé par Reimers et Gurevych en 2019, résout ce problème avec une architecture siamoise : deux copies du même réseau encodent indépendamment chaque phrase en un vecteur fixe. La comparaison se réduit alors à un simple calcul de similarité cosinus entre deux vecteurs, opération qui ne prend que quelques microsecondes :

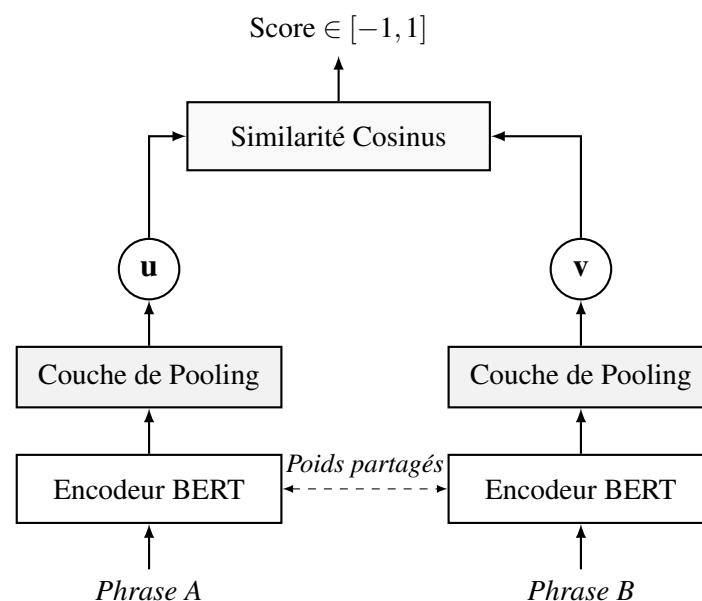


FIGURE 2.1 – Architecture siamoise de Sentence-BERT (SBERT) pour la comparaison sémantique

$$\text{Sim}_{\cos}(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \sqrt{\sum_{i=1}^d v_i^2}} \quad (2.1)$$

où $\mathbf{u}$ et $\mathbf{v}$ sont les vecteurs d’embedding de dimension d , et le résultat est compris entre -1 (sens opposé) et 1 (sens identique).

2.1.4 Choix du Modèle : paraphrase-multilingual-MiniLM-L12-v2

Pour *The Communium*, le choix du modèle d’embedding devait répondre à trois contraintes : supporter le **français, l’arabe et l’anglais** (les trois langues de la plateforme), être suffisamment **léger** pour tourner sur un serveur sans GPU dédié, et produire des vecteurs de qualité.

Le modèle paraphrase-multilingual-MiniLM-L12-v2, issu de la bibliothèque Sentence-Transformers, coche ces trois cases. Il prend en charge plus de 50 langues, produit des vecteurs de **384 dimensions** normalisés L2, et offre un bon compromis entre qualité et performance. Sa taille réduite (environ 470 Mo) permet un chargement en mémoire raisonnable, et son inférence ne nécessite pas de GPU.

2.1.5 Indexation Vectorielle : pgvector

Stocker des vecteurs ne suffit pas : il faut pouvoir les interroger efficacement. Plusieurs solutions existent sur le marché : **Pinecone** (service cloud propriétaire), **Weaviate** (base vectorielle dédiée), **Milvus** (open source, orienté haute performance), ou encore **pgvector**, une extension de PostgreSQL.

Pour notre projet, **pgvector** présentait un avantage décisif : il s’intègre directement dans PostgreSQL, la base de données déjà utilisée par le backend. Pas besoin d’ajouter un service supplémentaire dans l’infrastructure, pas de synchronisation à gérer entre deux systèmes. L’extension fournit un type natif `vector(384)`, l’opérateur de distance cosinus (`<=>`), et la possibilité de créer des index **HNSW** (*Hierarchical Navigable Small World*) pour accélérer les recherches approximatives sur de grands volumes.

2.2 Vision par Ordinateur et OCR

L’un des objectifs centraux de *The Communium* est d’automatiser la vérification d’identité (**KYC**) en lisant automatiquement la carte d’identité nationale marocaine (**CNIE**). Cette tâche relève de deux disciplines complémentaires : le prétraitement d’image et la reconnaissance optique de caractères.

2.2.1 Prétraitement d'Image

Une photo de carte d'identité nationale prise par un téléphone est rarement parfaite : elle peut être inclinée, mal éclairée, floue, ou prise sous un angle. Avant de tenter d'en lire le texte, il faut la préparer. Les techniques classiques de prétraitement incluent :

- **Conversion en niveaux de gris** : Réduit la complexité de l'image en supprimant l'information de couleur.
- **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) : Rehausse le contraste local, utile lorsque l'éclairage est inégal sur la carte.
- **Seuillage binaire** : Transforme l'image en noir et blanc pur. Le seuillage d'Otsu détermine automatiquement le seuil optimal, tandis que le seuillage adaptatif gère les variations locales de luminosité.
- **Détection de contours** (Canny) : Identifie les bords dans l'image, permettant de repérer la forme rectangulaire de la carte.
- **Transformation de perspective** : Une fois les quatre coins de la carte détectés, la fonction `warpPerspective` d'OpenCV applique une homographie pour « redresser » la carte, comme si elle avait été scannée à plat.

Un critère géométrique simple permet de filtrer les contours : la **CNIE** a un ratio d'aspect proche de celui d'une carte bancaire, soit environ 1.586. Ce filtre élimine les faux positifs lors de la détection de contours.

2.2.2 Reconnaissance Optique de Caractères

Une fois l'image prétraitée et la carte redressée, il faut en extraire le texte. Deux outils dominant le marché open source :

- **Tesseract** (Google) : Le moteur OCR le plus ancien et le plus répandu. Il fonctionne bien sur du texte imprimé régulier, mais ses performances chutent sur des documents multilingues ou lorsque le texte est mélangé à des éléments graphiques.
- **EasyOCR** (JaidedAI) : Un moteur OCR basé sur le Deep Learning qui prend en charge plus de 80 langues, dont le français, l'anglais et l'arabe. Sa capacité à traiter simultanément plusieurs scripts sur un même document en fait un choix naturel pour la **CNIE** marocaine, qui contient du texte en français et en arabe.

Pour notre cas d'usage, **EasyOCR** a été retenu pour sa gestion native du trilinguisme et ses performances supérieures sur des documents d'identité réels, sans nécessiter de fine-tuning.

2.3 Systèmes de Recommandation

La marketplace de *The Communium* doit proposer à chaque utilisateur des annonces pertinentes. Mais la pertinence seule ne suffit pas : si un utilisateur clique sur un téléphone, il ne veut pas voir vingt téléphones identiques en retour.

2.3.1 Filtrage Collaboratif vs. Filtrage par Contenu

Les deux grandes familles de systèmes de recommandation sont le **filtrage collaboratif** (« les utilisateurs similaires à toi ont aimé ceci ») et le **filtrage par contenu** (« cet article ressemble à ceux que tu as consultés »). Le filtrage collaboratif de base souffre du problème du *cold start* : pour un nouvel utilisateur ou une nouvelle annonce, il n’y a pas encore de données d’interaction. Le filtrage par contenu, lui, peut fonctionner dès la première annonce grâce aux embeddings textuels. C’est cette approche que nous avons retenue, en l’enrichissant d’un mécanisme de diversification.

2.3.2 Maximal Marginal Relevance (MMR)

Pour éviter l’effet de « bulle de filtres », nous nous sommes appuyés sur l’algorithme **Maximal Marginal Relevance** (MMR), proposé par Carbonell et Goldstein [carbonell1998](#). L’idée est de sélectionner itérativement les éléments qui sont à la fois *pertinents* par rapport au profil de l’utilisateur et *différents* des éléments déjà sélectionnés :

$$c^* = \arg \max_{c \in C \setminus S} \left[(1 - \lambda) \cdot \text{Sim}(c, U) - \lambda \cdot \max_{s \in S} \text{Sim}(c, s) \right] \quad (2.2)$$

où C est l’ensemble des candidats, S l’ensemble des éléments déjà sélectionnés, U le profil de l’utilisateur, et $\lambda \in [0, 1]$ contrôle l’équilibre entre pertinence et diversité. Dans notre implémentation, $\lambda = 0.3$, ce qui accorde **70 % à la pertinence et 30 % à la diversité**.

2.4 Détection d’Anomalies et Statistiques Robustes

Sur une marketplace, deux types de comportements frauduleux menacent la confiance des utilisateurs : les **prix aberrants** (un iPhone à 10 MAD ou un stylo à 50 000 MAD) et les **faux avis** (campagnes de notes 5 étoiles générées par des bots).

2.4.1 Z-Score Classique et ses Limites

La première approche pour détecter un prix anormal consiste à calculer son Z-Score : combien d'écarts-types le séparent de la moyenne de sa catégorie. Le problème, c'est que la moyenne et l'écart-type sont eux-mêmes sensibles aux valeurs extrêmes. Si un fraudeur poste dix annonces à des prix absurdes, la moyenne de la catégorie est tirée vers le haut, et le Z-Score des autres annonces est faussé.

2.4.2 Z-Score Modifié par la MAD

Pour contourner ce problème, Iglewicz et Hoaglin **iglewicz1993** proposent le **Z-Score modifié**, qui remplace la moyenne par la **médiane** et l'écart-type par la **MAD** (*Median Absolute Deviation*) :

$$\text{MAD} = \text{médiane} \left(|x_i - \tilde{X}| \right) \quad (2.3)$$

$$Z_{\text{mod}} = 0.6745 \times \frac{x_i - \tilde{X}}{\text{MAD}} \quad (2.4)$$

où $\tilde{X}$ est la médiane des prix et la constante 0.6745 assure la consistance avec la distribution normale. Un prix est signalé comme anomalie lorsque $|Z_{\text{mod}}| > 3.5$. L'avantage de cette méthode est qu'elle est **résistante aux outliers** : même si 20% des prix sont frauduleux, la médiane et la MAD restent stables.

2.4.3 Entropie de Shannon pour la Détection de Faux Avis

Pour les avis, la stratégie est différente. Un produit qui reçoit exclusivement des notes de 5/5 est suspect : dans la réalité, les avis authentiques présentent toujours une certaine dispersion. L'**entropie de Shannon shannon1948** mesure précisément cette dispersion :

$$H(X) = - \sum_{i=1}^5 P(x_i) \log_2 P(x_i) \quad (2.5)$$

où $P(x_i)$ est la proportion d'avis ayant la note i . L'entropie est maximale lorsque les notes sont uniformément réparties, et nulle lorsqu'elles sont toutes identiques. En normalisant par $\log_2(5)$, on obtient un score d'uniformité entre 0 et 1. Un score proche de 1 signale une distribution anormalement concentrée.

Une distribution de notes anormalement concentrée constitue ainsi un **signal d'alerte** parmi d'autres (régularité temporelle des dépôts, similarité textuelle entre avis, multiplicité des avis d'un même auteur). L'exploitation conjointe de ces signaux pour un détecteur de faux avis complet constitue

une piste d'évolution naturelle de la plateforme ; dans le cadre de ce mémoire, l'effort s'est concentré sur l'analyse de sentiment des avis authentiques, l'entropie fournissant ici un cadre théorique pour qualifier la fiabilité d'une distribution de notes.

2.5 Prédiction d'Attrition Client (Churn)

Pour une plateforme communautaire, perdre des utilisateurs actifs est un enjeu critique. Anticiper les départs permet de déclencher des actions de rétention avant qu'il ne soit trop tard.

2.5.1 Analyse RFM

Le modèle **RFM** est un classique du marketing analytique. Il segmente les utilisateurs selon trois dimensions : la **Récence** (date de dernière activité), la **Fréquence** (nombre d'actions sur la plateforme), et la **valeur Monétaire** (montant dépensé ou tokens utilisés). Chaque dimension reçoit un score de 1 à 5 par découpage en quintiles. Un utilisateur avec un score R faible, F faible et M faible est à haut risque de départ.

Dans notre adaptation, nous avons ajouté une quatrième dimension : l'**Ancienneté** (*Tenure*), qui mesure depuis combien de temps l'utilisateur est inscrit. Un utilisateur récent qui n'a jamais été actif n'a pas le même profil de risque qu'un utilisateur fidèle qui commence à décrocher.

2.5.2 Customer Lifetime Value (CLV)

La **CLV** estime la valeur financière totale qu'un utilisateur peut générer sur sa durée de vie restante. Nous utilisons un modèle DCF (*Discounted Cash Flow*) simplifié :

$$CLV = Rev_m \times \frac{1 - P_{\text{churn}_m}}{P_{\text{churn}_m} + d_m} \quad (2.6)$$

où Rev_m est le revenu mensuel moyen de l'utilisateur, P_{churn_m} sa probabilité mensuelle de départ, et d_m le taux d'actualisation mensuel. Cette formule permet de prioriser les actions de rétention : on cible en priorité les utilisateurs à forte CLV et à risque de churn élevé.

2.6 Classification Automatique et Analyse de Sentiment

Au-delà de la recherche et de la recommandation, deux tâches de traitement du langage naturel structurent l'expérience sur *The Communium* : le rangement automatique des annonces dans la bonne catégorie, et l'évaluation du ton des avis laissés par les utilisateurs. Ces deux fonctions partagent une

contrainte commune — le trilinguisme du contenu (français, arabe, anglais) — et une exigence forte : fonctionner sans disposer d’un corpus annoté spécifique au marché marocain.

2.6.1 Classification Zero-Shot par Similarité Sémantique

La classification de texte supervisée classique exige un jeu de données étiqueté pour chaque catégorie cible. Sur une marketplace dont les catégories évoluent et où aucune base annotée d’annonces marocaines n’existe, cette approche est inapplicable. La **classification zero-shot** contourne ce verrou en permettant d’attribuer un texte à une classe *jamais vue à l’entraînement*.

Deux grandes stratégies coexistent dans la littérature. La première s’appuie sur l’inférence en langage naturel (NLI) : Yin et al. **yin2019** reformulent la classification en un test d’implication entre le texte et une hypothèse du type « ce texte parle de *catégorie* ». La seconde, plus légère, exploite directement l’espace d’*embeddings* multilingue : le texte et chaque libellé de catégorie sont encodés en vecteurs, puis comparés par similarité cosinus. La catégorie retenue est celle dont le vecteur est le plus proche :

$$\hat{c} = \arg \max_{c \in \mathcal{C}} \text{Sim}_{\cos}(\mathbf{e}_{\text{texte}}, \mathbf{e}_c) \quad (2.7)$$

où $\mathcal{C}$ est l’ensemble des catégories candidates, $\mathbf{e}_{\text{texte}}$ le vecteur de l’annonce et $\mathbf{e}_c$ celui du libellé de la catégorie c . C’est cette seconde approche que nous avons retenue : elle réutilise le modèle d’embeddings déjà chargé (MiniLM-L12-v2), n’ajoute aucune dépendance, comprend nativement les trois langues de la plateforme, et s’adapte instantanément à toute nouvelle catégorie sans réentraînement. Sur des taxonomies produit usuelles, ce type de classification atteint typiquement de 75 à 85 % de précision sans aucune donnée d’entraînement.

2.6.2 Analyse de Sentiment des Avis

L’analyse de sentiment vise à déterminer la polarité (positive, négative ou neutre) d’un texte. Deux familles d’approches dominent.

Les méthodes **lexicales** reposent sur un dictionnaire associant à chaque mot un score de polarité, ajusté par des règles linguistiques (négations, intensificateurs, ponctuation). Le modèle de référence est **VADER hutto2014**, conçu pour les textes courts et informels. Il agrège les scores des mots en un score composite normalisé entre -1 et $+1$:

$$\text{compound} = \frac{\sum_i s_i}{\sqrt{(\sum_i s_i)^2 + \alpha}} \quad (2.8)$$

où s_i est le score du i -ème terme et α une constante de normalisation. L'avantage de l'approche lexicale est sa rapidité et son absence de coût d'inférence. Sa faiblesse est sa dépendance à la couverture du dictionnaire : VADER est pensé pour l'anglais, et un avis rédigé en darija ou en français peut ne contenir aucun terme connu.

Les méthodes par **embeddings** corrigent cette limite : en projetant l'avis dans l'espace vectoriel sémantique, on peut mesurer sa proximité avec des phrases-ancres positives et négatives, même sans correspondance lexicale exacte. Le sentiment se déduit alors de l'écart entre ces deux distances.

Pour *The Communium*, nous avons retenu une **approche hybride** : un lexique trilingue (FR/AR/EN) constitue le chemin rapide, et lorsqu'un avis contient trop peu de termes connus, le système bascule sur un calcul de similarité par embeddings comme repli. Cette combinaison conjugue la rapidité du lexique et la robustesse sémantique des embeddings, tout en restant entièrement locale.

2.7 Estimation des Délais de Livraison

Lors d'une transaction sur la marketplace, afficher une estimation du délai de livraison (*ETA* — *Estimated Time of Arrival*) réduit l'incertitude de l'acheteur et améliore la confiance. Idéalement, cette estimation serait apprise à partir de l'historique réel des livraisons via un modèle de régression. Or, au lancement de la plateforme, ces données n'existent pas encore : c'est le problème classique du *cold start* appliqué à la prédiction.

En l'absence de données historiques, l'approche la plus robuste consiste à construire un **modèle de référence (baseline) par ingénierie de caractéristiques** : plutôt que d'apprendre des coefficients, on combine des facteurs métier connus pour influencer le délai. Le délai estimé résulte du produit d'un délai de base, dépendant de la catégorie, et d'une série de multiplicateurs :

$$ETA = (\text{base}_{\text{cat}} \times m_{\text{dist}} \times m_{\text{état}} \times m_{\text{vendeur}}) + \delta_{\text{temporel}} \quad (2.9)$$

où base_{cat} est le délai de référence de la catégorie (d'un service immatériel quasi instantané à un véhicule de plusieurs jours), m_{dist} tient compte des zones géographiques (les grandes villes — Casablanca, Rabat — étant mieux desservies que les zones rurales), $m_{\text{état}}$ et m_{vendeur} ajustent selon l'état du bien et le sérieux du vendeur (nombre d'annonces déjà publiées), et δ_{temporel} ajoute un retard forfaitaire selon l'heure et le jour (soirée, nuit, week-end).

Cette formulation présente deux avantages : elle est interprétable — chaque facteur a un sens métier clair — et elle fournit immédiatement une estimation exploitable. Elle constitue surtout une **fondation évolutive** : dès que la plateforme aura accumulé un historique de livraisons réelles, ces multiplicateurs heuristiques pourront être remplacés par les coefficients d'un modèle de régression entraîné, sans rien

changer à l'architecture du service.

2.8 Benchmark des Technologies

Le développement de *The Communium* a nécessité des choix technologiques à chaque couche de l'architecture. Le tableau 2.1 synthétise les alternatives évaluées et justifie les choix retenus.

TABLE 2.1 – Benchmark des technologies évaluées pour chaque couche

Couche	Choix retenu	Alternatives	Justification
Frontend	Next.js 14	Nuxt 3, SvelteKit	App Router, SSR natif, écosystème React mature, support i18n
Backend	NestJS	Express, Fastify	Architecture modulaire, injection de dépendances, Swagger intégré, TypeScript natif
Service IA	FastAPI	Flask, Django	Asynchrone, typage Pydantic, documentation auto, performance élevée
BDD	PostgreSQL	MongoDB, MySQL	Relationnel robuste, extension pgvector, support JSONB, maturité
Vecteurs	pgvector	Pinecone, Weaviate	Pas de service externe, même BDD que les données métier, index HNSW
Recherche	Meilisearch	Elasticsearch, Algolia	Léger, rapide, tolérant aux fautes de frappe, API simple, open source
Cache	Redis	Memcached	Structures avancées, TTL natif, pub/sub pour temps réel
OCR	EasyOCR	Tesseract, Google Vision	Trilingue fr/ar/en, Deep Learning, pas d'API externe payante
Embeddings	MiniLM-L12-v2	all-mpnet, LaBSE	50+ langues, 384 dim (léger), bon compromis qualité/taille

Un point mérite d'être souligné : le choix de **séparer le backend transactionnel (NestJS/TypeScript) du service IA (FastAPI/Python)** n'est pas anodin. Les bibliothèques de Machine Learning (sentence-transformers, OpenCV, numpy) sont quasi exclusivement disponibles en Python, tandis que l'écosystème web TypeScript offre une meilleure intégration avec le frontend et les outils comme Prisma ou Socket.IO. Plutôt que de forcer un langage unique, nous avons opté pour une architecture polyglotte

qui tire le meilleur de chaque écosystème.

Conclusion

Ce chapitre a posé les bases théoriques nécessaires à la compréhension de notre solution. La recherche sémantique par embeddings, la vision par ordinateur pour l'OCR, la recommandation diversifiée par MMR, la détection d'anomalies par statistiques robustes et la prédiction de churn par analyse RFM constituent les piliers algorithmiques de *The Communium*. Le benchmark technologique a permis de justifier chaque choix d'outil en fonction des contraintes spécifiques du projet : trilinguisme, absence de GPU, et intégration avec une base PostgreSQL existante. Le chapitre suivant détaille comment ces briques théoriques ont été assemblées dans une architecture cohérente.

Chapitre 3

Conception et Architecture

Introduction

Le chapitre précédent a présenté les fondements théoriques des composants intelligents de la plateforme. Celui-ci explique comment ils ont été assemblés. La présentation part de la vue d'ensemble, l'organisation en monorepo et l'architecture en trois tiers, puis entre dans le détail de la pile technologique, du modèle de données PostgreSQL avec *pgvector* et de la modélisation UML. La dernière partie formalise l'architecture interne du microservice d'Intelligence Artificielle, où chaque algorithme vu au chapitre 2 trouve sa traduction concrète. Les mécanismes de sécurité, qui concernent l'ensemble du système, sont traités en fin de chapitre.

3.1 Architecture Globale du Système

Trois besoins très différents se rencontrent dans *The Communion*. Le premier est un frontend riche et trilingue. Le deuxième est une logique métier transactionnelle qui couvre les profils, la marketplace, la messagerie en temps réel et les paiements. Le troisième regroupe des traitements d'Intelligence Artificielle coûteux en calcul. Vouloir les faire cohabiter dans un langage et un processus uniques aurait imposé des compromis sur chacun. L'architecture retenue les sépare en trois tiers, tout en réunissant leur code dans un même **monorepo**.

3.1.1 Organisation en Monorepo (PNPM Workspaces + Turborepo)

Tout le code réside dans un dépôt unique. **PNPM Workspaces** y gère le partage des dépendances, tandis que **Turborepo** orchestre les tâches de build et met leurs résultats en cache. On y gagne sur plusieurs plans : les versions restent cohérentes entre applications, les types et les schémas se partagent sans duplication, et le cache incrémental raccourcit nettement les compilations. Le dépôt contient trois applications et un paquet partagé :

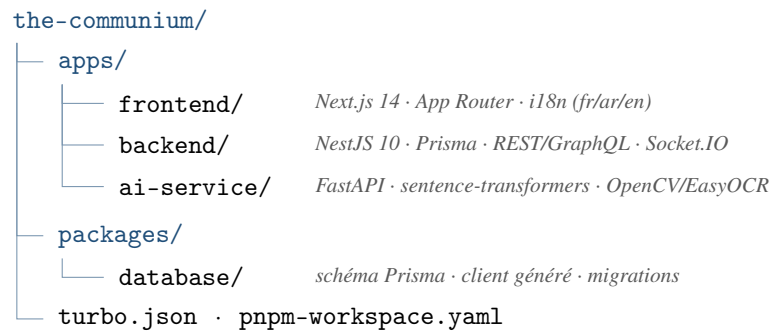


FIGURE 3.1 – Organisation du monorepo PNPM + Turborepo

Le paquet `packages/database` joue un rôle particulier. Il contient l'unique schéma Prisma et le client généré, que le backend comme les scripts de migration importent. Le modèle de données ne peut donc pas diverger d'une application à l'autre.

3.1.2 Architecture Micro-services en Trois Tiers

Les échanges entre applications passent par des interfaces réseau précises. Le **frontend** Next.js ne communique qu'avec le **backend** NestJS, qui orchestre le reste : logique métier, accès à la base, authentification, sessions temps réel. Quand une tâche relève de l'Intelligence Artificielle, le backend la confie au **service IA** FastAPI par un appel HTTP interne, récupère le résultat (un embedding, un score, des données extraites) et l'enregistre dans PostgreSQL.

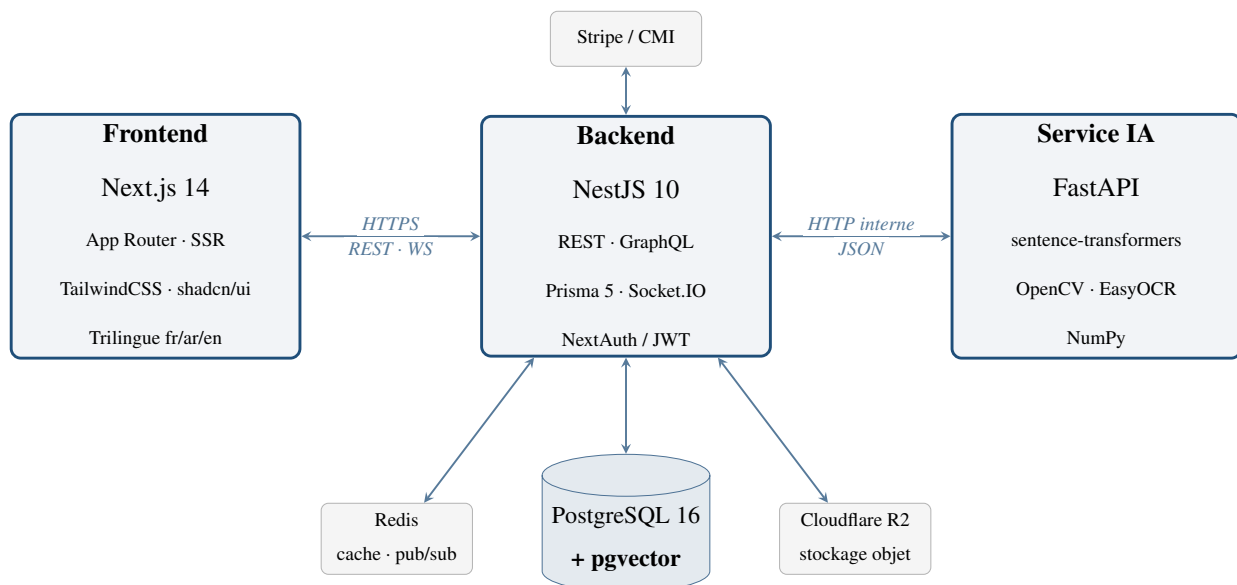


FIGURE 3.2 – Architecture micro-services en trois tiers de The Communium

Ce découplage a des effets très concrets. On peut déployer et redimensionner chaque service de son côté : ajouter des instances du service IA, par exemple, sans rien changer au backend. Si le service IA tombe, les fonctions transactionnelles continuent de répondre, en mode dégradé. Et chaque partie

du système, donc chaque volet de mon travail, utilise la pile la mieux adaptée à son besoin.

3.1.3 Justification de la Séparation NestJS / FastAPI

La séparation entre le backend transactionnel et le service IA mérite qu'on s'y arrête, car elle structure tout le reste. La première raison tient à l'écosystème : les bibliothèques de Machine Learning matures comme sentence-transformers, OpenCV ou NumPy existent presque uniquement en Python, là où Prisma, Socket.IO et NextAuth sont plus aboutis en TypeScript. La deuxième tient à la performance et à l'isolation. Les modèles d'IA, chargés une fois en mémoire, consomment beaucoup de CPU et de RAM ; les garder à l'écart évite qu'un encodage coûteux ralentisse les requêtes transactionnelles. La troisième tient à la montée en charge : le transactionnel et l'IA n'ont pas les mêmes profils de trafic, et les séparer permet de dimensionner chacun pour ce qu'il fait. Cette approche polyglotte ne cherche pas un langage de compromis ; elle laisse chaque service sur son terrain le plus solide.

3.2 Pile Technologique

Le tableau [3.1](#) reprend la pile complète, couche par couche, avec le rôle de chaque technologie.

TABLE 3.1 – Pile technologique de The Communium par couche

Couche	Technologie	Rôle dans le système
Frontend	Next.js 14	Interfaces, App Router, rendu SSR, internationalisation
Backend	NestJS 10	Logique métier, orchestration, API REST/GraphQL
ORM	Prisma 5	Accès typé à la base, migrations, client partagé
Service IA	FastAPI 0.115	Exposition des modèles d’IA via endpoints HTTP
Serveur ASGI	Uvicorn	Exécution asynchrone du service IA
Embeddings	sentence-transformers 3.0	Vectorisation sémantique multilingue (384 dim)
Vision / OCR	OpenCV 4.10 · EasyOCR 1.7	Détection de carte et extraction de texte CNIE
Calcul	NumPy 1.26	Opérations vectorielles et statistiques robustes
Validation	Pydantic v2	Schémas typés des requêtes/réponses IA
Base de données	PostgreSQL 16 + pgvector	Stockage relationnel et vectoriel unifié
Recherche	Meilisearch	Recherche plein texte tolérante aux fautes
Cache / Temps réel	Redis	Cache, TTL, pub/sub pour la messagerie
Stockage objet	Cloudflare R2	Images, pièces jointes, documents KYC
Paieement	Stripe / CMI	Transactions et économie de tokens
Conteneurisation	Docker Compose	Orchestration locale et déploiement des services

3.3 Architecture des Données

Le modèle de données porte toute la plateforme. Construit avec Prisma sur PostgreSQL 16, il couvre les domaines fonctionnels et intègre directement les vecteurs dont la recherche sémantique a besoin.

3.3.1 Schéma Prisma et Domaines Fonctionnels

Le schéma réunit une soixantaine de modèles, répartis en sept domaines : **Identité** (utilisateurs, profils, vérification KYC/KYB), **Commerce** (annonces, commandes, enchères, paiements, tokens), **Communauté** (publications, commentaires, événements, forum), **Mentorat** (profils mentors, sessions, réservations), **Messagerie** (conversations, messages temps réel), **IA** (embeddings, scores, résultats

d’analyse) et **Modération** (signalements, journaux d’audit, rôles). La figure 3.3 en donne la vue d’ensemble avec les principales relations.

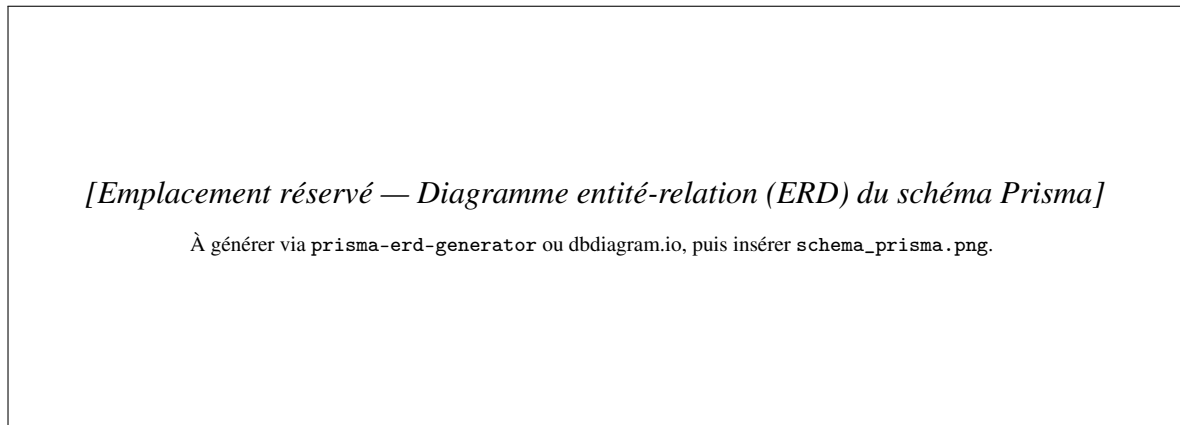


FIGURE 3.3 – Vue d’ensemble des domaines du schéma de données (Prisma)

3.3.2 Le Modèle Utilisateur, Point Central

Le modèle User est au centre du schéma. Une trentaine de relations le relie à presque tous les autres domaines : annonces publiées, commandes, messages, sessions de mentorat, publications. Il porte aussi les champs qui gouvernent les droits et le cycle de vie du compte : le rôle (`role`) pour le contrôle d’accès, le statut (`status`, actif, suspendu ou en attente) et l’état de vérification (`isVerified`), fixé à l’issue du KYC. La figure 3.4 illustre cette position centrale.

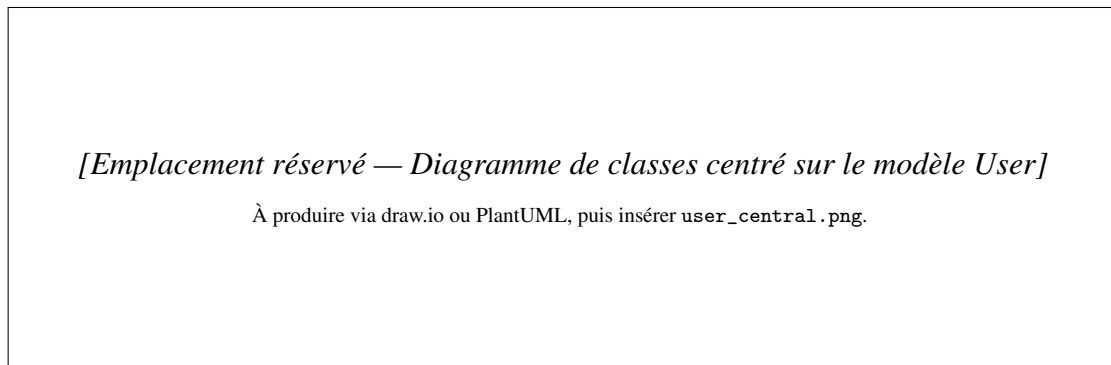


FIGURE 3.4 – Le modèle User et ses relations principales

3.3.3 Intégration de pgvector avec Prisma

La recherche sémantique oblige à stocker, à côté des données métier, des vecteurs d’embedding de dimension 384. Plutôt que d’ajouter une base vectorielle séparée, le projet s’appuie sur l’extension `pgvector` au sein même de PostgreSQL (voir chapitre 2). Les tables concernées, annonces et profils mentors, reçoivent une colonne de type `vector(384)`. Prisma ne connaît pas ce type : la colonne est donc déclarée en type non géré et remplie par des requêtes SQL brutes au moment d’écrire l’embedding.

Les recherches de voisinage emploient l'opérateur de distance cosinus $\leq$ et un index **HNSW** pour rester rapides sur de gros volumes. La donnée garde ainsi une source unique, avec un seul système à déployer et à sauvegarder.

3.3.4 Stratégie de Cache (Redis)

Redis sert à deux choses. Comme cache, il garde un moment les résultats coûteux mais stables : embeddings souvent demandés, sorties de recommandation, suggestions de prix. Chaque entrée a une durée de vie (*TTL*) calée sur sa volatilité, ce qui soulage le service IA et la base. Comme bus de messages (*pub/sub*), il diffuse en temps réel les événements de messagerie et de notification entre les instances du backend, sans quoi les connexions WebSocket ne pourraient pas se répartir sur plusieurs instances.

3.4 Modélisation Fonctionnelle (UML)

La modélisation UML fixe les interactions entre les acteurs et le système avant d'écrire le code. Cette partie présente d'abord les cas d'utilisation, module par module, puis les scénarios dynamiques importants en diagrammes de séquence, et enfin le processus d'onboarding en diagramme d'activité.

3.4.1 Acteurs du Système

Quatre acteurs principaux utilisent la plateforme. Le **Visiteur** n'est pas authentifié et accède seulement à la vitrine et à l'inscription. Le **Membre** est un entrepreneur, un freelance ou une entreprise déjà vérifié. Le **Mentor** est un membre qui dispose d'un profil de mentorat. L'**Administrateur** prend en charge la modération, la gestion des comptes et la supervision. Le service IA intervient comme acteur système secondaire, appelé par le backend.

3.4.2 Diagrammes de Cas d'Utilisation

Module Marketplace

Ce module gère la publication d'annonces, la recherche et le filtrage, le panier, la commande, le paiement et les enchères. L'IA y apporte la classification automatique, la suggestion de prix et la recommandation diversifiée.

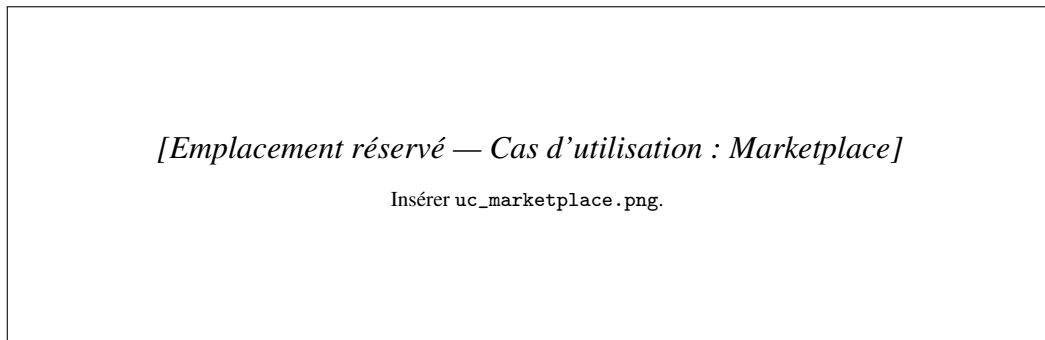


FIGURE 3.5 – Diagramme de cas d'utilisation — Module Marketplace

Module Mentorat

Le membre recherche un mentor, consulte les profils proposés par l'appariement intelligent, réserve une session et laisse un avis. Le mentor gère sa disponibilité et ses créneaux.

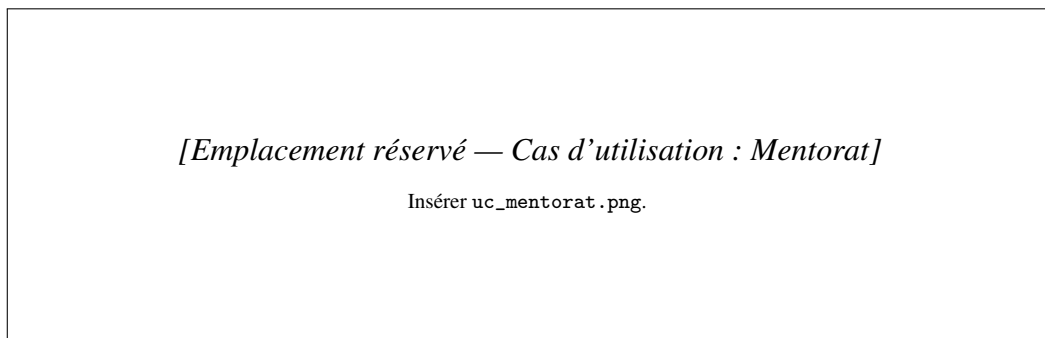


FIGURE 3.6 – Diagramme de cas d'utilisation — Module Mentorat

Module KYC / KYB

Le membre téléverse sa pièce d'identité (CNIE) ou ses documents d'entreprise ; le système extrait et vérifie les données, puis met à jour l'état de vérification du compte. L'administrateur peut réviser les cas ambigus.

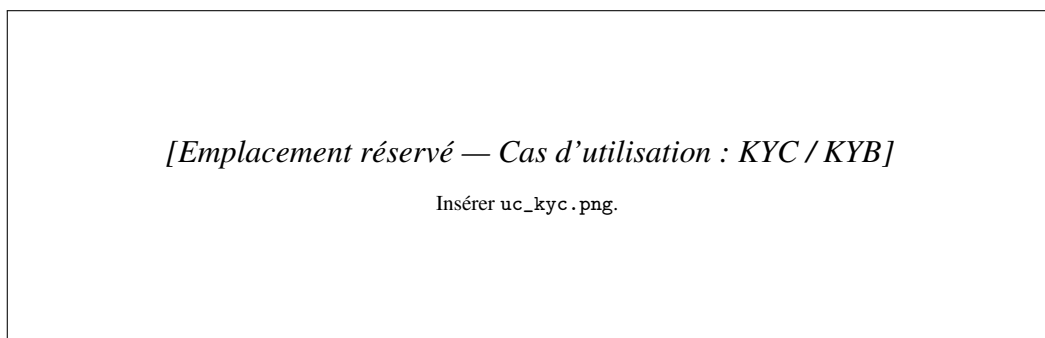


FIGURE 3.7 – Diagramme de cas d'utilisation — Module KYC / KYB

Module IA

Ce module transversal regroupe les services sollicités par les autres modules : génération d’embeddings, similarité, appariement de mentors, classification, analyse de sentiment, détection d’anomalies, prédiction de churn, recommandation et OCR.

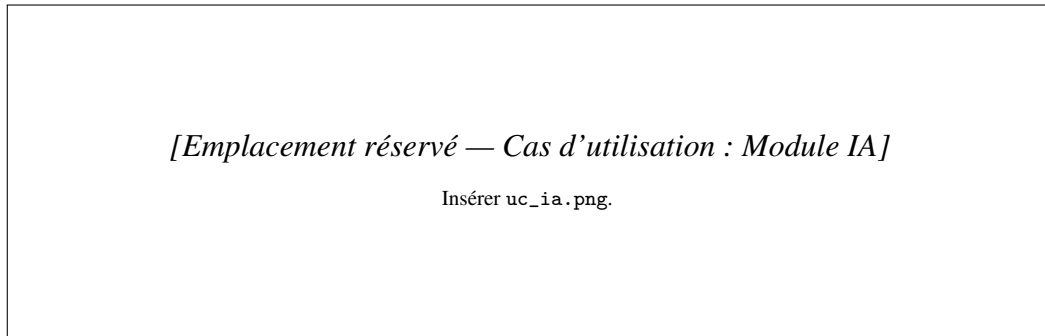


FIGURE 3.8 – Diagramme de cas d’utilisation — Module IA

3.4.3 Diagrammes de Séquence

Création d’Annonce Enrichie par l’IA

Quand un membre publie une annonce, le frontend envoie son contenu au backend, qui sollicite le service IA en plusieurs étapes : une classification *zero-shot* pour la catégorie et les tags, une suggestion de prix à partir des comparables, puis la génération de l’embedding, ensuite stocké via *pgvector*. Le membre récupère une fiche déjà remplie et enrichie.

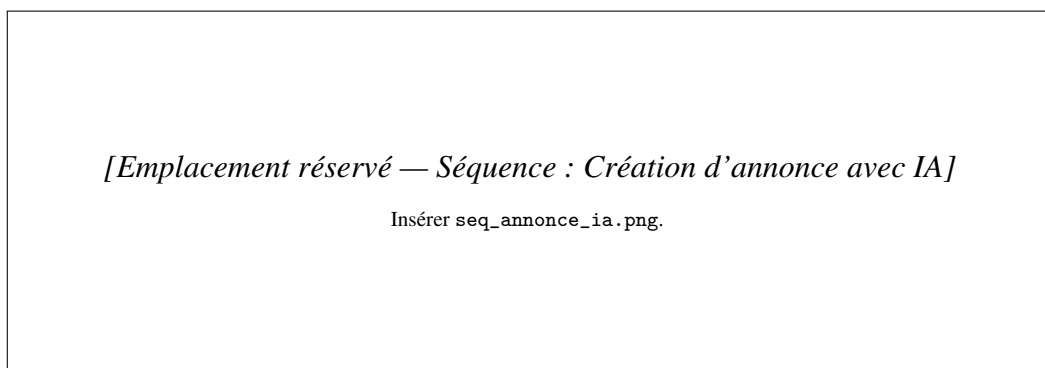


FIGURE 3.9 – Diagramme de séquence — Création d’annonce enrichie par l’IA

Vérification KYC par OCR de la CNIE

Le membre téléverse une photo de sa carte. Le backend la transmet au service IA, qui déroule le pipeline de vision et d’OCR (détaillé en section 3.5.10) et renvoie les champs extraits avec un score de confiance. Le backend met alors à jour le profil et l’état de vérification.

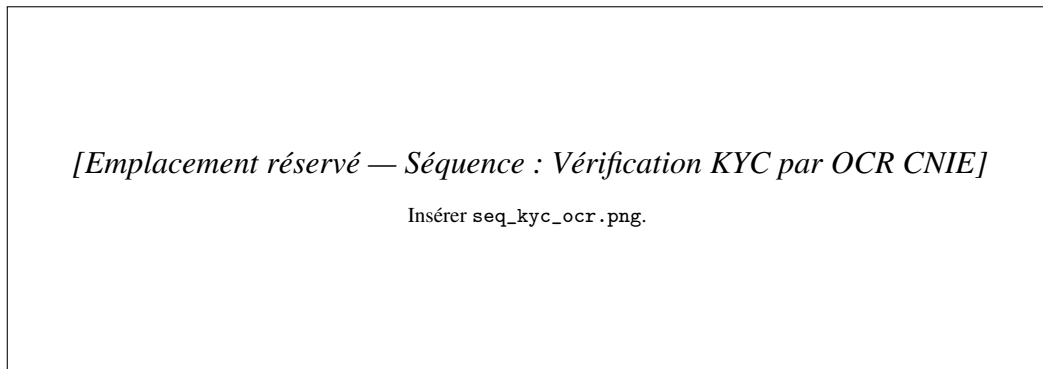


FIGURE 3.10 – Diagramme de séquence — Vérification KYC par OCR de la CNIE

3.4.4 Diagramme d'Activité de l'Onboarding

Le parcours d'intégration enchaîne plusieurs étapes, dont certaines conditionnent la suite : choix du type de compte, saisie des coordonnées, vérification KYC/KYB, choix de l'abonnement et finalisation du profil. La figure 3.11 en formalise les transitions et les points de décision.



FIGURE 3.11 – Diagramme d'activité — Processus d'intégration (Onboarding)

3.5 Architecture du Service d'Intelligence Artificielle

Le microservice IA expose dix points de terminaison REST, chacun pour une fonction précise. Ils partagent une même base : un service d'embeddings chargé une seule fois en mémoire (patron *singleton*) et réutilisé par les modules qui en ont besoin. La figure 3.12 montre son organisation interne.

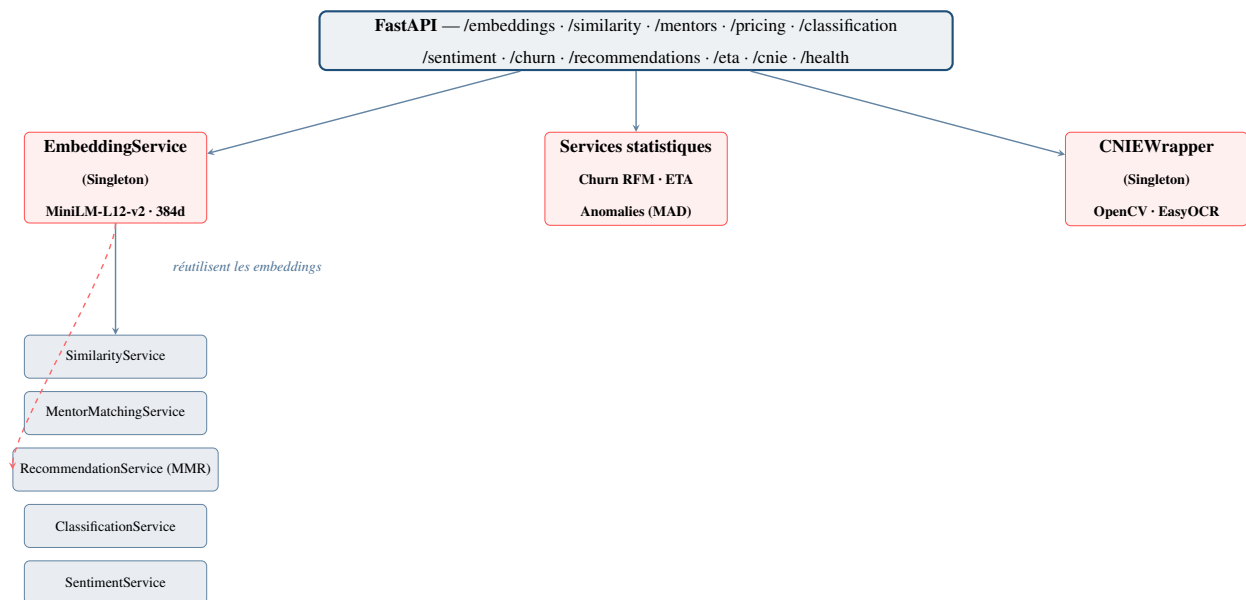


FIGURE 3.12 – Architecture interne du microservice d’Intelligence Artificielle

3.5.1 Génération d’Embeddings Sémantiques

Le service d’embeddings transforme un texte en un vecteur dense de 384 dimensions, normalisé L2, avec le modèle paraphrase-multilingual-MiniLM-L12-v2. Avant l’encodage, l’objet métier est mis à plat sous forme de texte : pour une annonce, on concatène le titre, la description, la catégorie et les tags ; pour un mentor, l’intitulé, la biographie et les listes d’expertises et de secteurs. Le chargement du modèle étant lent, il n’a lieu qu’une fois au démarrage, et toutes les fonctions en aval réutilisent la même instance. L’encodage d’un texte prend alors quelques millisecondes.

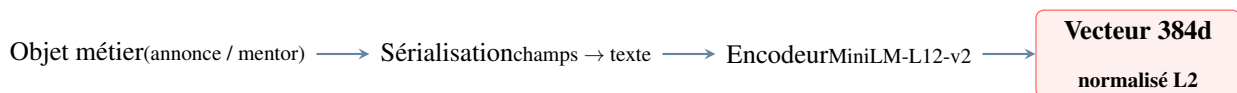


FIGURE 3.13 – Pipeline de génération d’un embedding

3.5.2 Moteur de Similarité Vectorielle

La similarité se calcule par produit scalaire entre vecteurs déjà normalisés, ce qui revient à la similarité cosinus de l’équation 2.1. Pour une annonce donnée, le service mesure ce score face à chaque candidat, classe les résultats par ordre décroissant et renvoie les *top-K*. L’opération est purement arithmétique et déterministe ; elle alimente la fonction « Vous aimerez aussi ».

3.5.3 Appariement Intelligent de Mentorat

L’appariement entre mentor et mentoré ne peut pas reposer sur la seule similarité sémantique : un mentor pertinent sur le fond mais mal noté ou indisponible reste un mauvais résultat. Plusieurs signaux

sont donc combinés en un **score hybride** :

$$S_{\text{mentor}} = 0.55 \text{sim}_{\text{cos}} + 0.25 r_{\text{norm}} + 0.12 e_{\text{log}} + 0.08 s_{\text{log}} \quad (3.1)$$

où sim_{cos} est la similarité sémantique entre les profils, r_{norm} la note du mentor normalisée (min-max) dans le pool de candidats, et e_{log} , s_{log} respectivement l'expérience (en années) et le nombre de sessions, comprimés logarithmiquement pour atténuer l'effet des valeurs extrêmes :

$$v_{\text{log}} = \frac{\log(1 + \min(v, \text{cap}))}{\log(1 + \text{cap})}, \quad \text{cap}_{\text{exp}} = 20, \quad \text{cap}_{\text{sessions}} = 200 \quad (3.2)$$

Un mentor indisponible voit enfin son score multiplié par 0.6. La pondération met volontairement la pertinence du domaine au premier plan (55 %), sans ignorer la qualité attestée par la note (25 %) ni l'expérience et les sessions passées (20 % à elles deux).

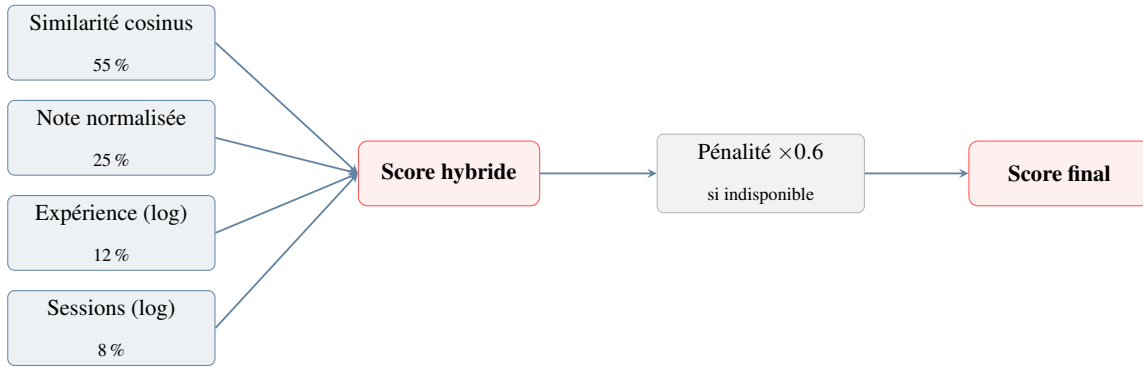


FIGURE 3.14 – Calcul du score hybride d'appariement de mentorat

3.5.4 Suggestion de Prix et Détection d'Anomalies

Ce module poursuit deux objectifs complémentaires : aider le vendeur à fixer un prix juste, et signaler les prix manifestement anormaux.

Suggestion de prix. Dès qu'au moins trois prix comparables existent, le service retire les valeurs aberrantes par l'écart interquartile (IQR) et ne garde que les prix situés entre $Q_1 - 1.5 \text{IQR}$ et $Q_3 + 1.5 \text{IQR}$. Il en tire trois percentiles, P_{20} , P_{50} et P_{80} , qui donnent le prix plancher, le prix recommandé et le prix plafond, avant d'appliquer un multiplicateur lié à l'état du bien (tableau 3.2). La confiance augmente avec le nombre de comparables : $\text{conf} = \min(0.95, 0.50 + 0.05n)$. Quand les comparables manquent (*cold start*), une heuristique de repli propose une fourchette de $\pm 25\%$ autour d'un prix de base, avec une confiance basse assumée.

TABLE 3.2 – Multiplicateurs de prix selon l'état du bien

État	Multiplicateur
Neuf (NEW)	×1.30
Comme neuf (LIKE_NEW)	×1.15
Bon (GOOD)	×1.00
Correct (FAIR)	×0.78
Médiocre (POOR)	×0.55

Détection d'anomalies. Pour repérer un prix suspect, le service croise le Z-Score classique (seuil $|z| > 2.5$) et le Z-Score modifié par la MAD (équations 2.3 et 2.4, seuil $|Z_{\text{mod}}| > 3.5$). Le second s'appuie sur la médiane et tient bon même quand une bonne partie des prix est frauduleuse, là où la moyenne du premier se laisserait tromper.

3.5.5 Classification Zero-Shot et Extraction de Tags

La catégorisation d'une annonce se fait sans aucune donnée d'entraînement, par similarité sémantique (équation 2.7). Le texte de l'annonce, où le titre est compté trois fois pour peser davantage, est encodé en même temps que chaque libellé de catégorie ; la catégorie dont le vecteur est le plus proche l'emporte, et la similarité maximale sert de score de confiance. En parallèle, une extraction de tags inspirée de la fréquence des termes (titre compté trois fois, puis description) retient les six mots les plus saillants. Un détecteur d'urgence, fondé sur des mots-clés trilingues, range enfin l'annonce en niveau haut, moyen ou bas.

3.5.6 Analyse de Sentiment des Avis

L'analyse de sentiment reprend la stratégie hybride du chapitre 2. Un premier niveau, lexical, parcourt l'avis, repère les termes du lexique trilingue (FR/AR/EN), traite les négations et les intensificateurs (par exemple ×1.4 pour « très » et ×1.5 pour « extrêmement »), puis agrège le tout en un score composite normalisé (équation 2.8). Ce résultat est conservé si au moins deux termes du lexique sont reconnus. Sinon, un second niveau prend le relais avec les embeddings : le sentiment se lit dans l'écart entre la proximité de l'avis avec une phrase-ancre positive et avec une phrase-ancre négative. Le score final se range en trois classes : positif (≥ 0.05), négatif (≤ -0.05) ou neutre.

3.5.7 Prédiction d'Attrition (Churn) et Calcul de CLV

Le risque de départ s'estime avec une variante du modèle RFM, à laquelle s'ajoute une dimension d'ancienneté (*Tenure*) :

$$\text{RFM}_{\text{brut}} = 0.40R + 0.30F + 0.20M + 0.10T \quad (3.3)$$

où R , F , M et T sont des scores de 1 à 5 (récence, fréquence, valeur monétaire, ancienneté). Après ajout d'un bonus d'adhésion et normalisation, le score de churn vaut $1 - \text{RFM}_{\text{normalisé}}$, puis est classé en risque faible (< 0.35), moyen (0.35 à 0.65) ou élevé (≥ 0.65). La valeur vie client (CLV) est ensuite estimée par un modèle de flux actualisés : à partir du score de churn, on déduit un taux de désabonnement mensuel et un taux de rétention, et la CLV agrège le revenu mensuel futur actualisé (cf. équation 2.6). Croiser CLV élevée et risque élevé permet de prioriser les actions de rétention.

3.5.8 Prédiction des Délais de Livraison (ETA)

À défaut d'historique de livraisons, le délai estimé est produit par ingénierie de caractéristiques (équation 2.9) : un délai de base par catégorie est ajusté par des multiplicateurs de distance (zones de villes), d'état du bien et de sérieux du vendeur, auxquels s'ajoute un retard forfaitaire selon l'heure et le jour. Sept caractéristiques sont ainsi combinées. La confiance de l'estimation décroît à mesure que des informations manquent (ville du vendeur ou de l'acheteur, catégorie inconnue), avec un plancher garanti. Ce module est conçu comme une base évolutive, destinée à être remplacée par un modèle de régression entraîné dès que des données réelles seront disponibles.

3.5.9 Recommandation Diversifiée via MMR

Le moteur de recommandation équilibre pertinence et diversité grâce à l'algorithme *Maximal Marginal Relevance* (équation 2.2). À partir du profil de goût de l'utilisateur, il sélectionne itérativement les annonces qui maximisent la pertinence tout en pénalisant la ressemblance avec les éléments déjà retenus. Le paramètre $\lambda = 0.3$ accorde 70 % du poids à la pertinence et 30 % à la diversité, évitant ainsi l'effet de bulle de filtres.

3.5.10 Extraction de Données CNIE par OCR

L'extraction des données de la carte nationale d'identité électronique suit un pipeline en quatre étapes (figure 3.5.10). La première, de **vision par ordinateur**, rehausse le contraste (CLAHE), combine plusieurs méthodes de seuillage (Otsu, adaptatif) et la détection de contours (Canny), applique des

opérations morphologiques, puis score chaque contour selon son ratio d'aspect, sa solidité, son aire et sa rectangularité ; le meilleur candidat (ratio idéal proche de 1.586) est redressé par transformation de perspective. Une validation de confiance (≥ 15) conditionne la suite : en cas d'échec, l'image entière est transmise telle quelle à l'étape suivante (*fallback*). La deuxième étape applique l'**OCR** (EasyOCR, configuré pour les scripts présents sur la carte) pour produire le texte brut. La troisième réalise le **parsing** par expressions régulières (numéro de CIN, dates) complété par des heuristiques de repli sur les libellés. La quatrième assemble un **profil structuré** (nom, prénom, date et lieu de naissance, adresse, nationalité).

3.6 Sécurité

La sécurité est traitée de façon transverse. L'**authentification** repose sur NextAuth et des jetons JWT, avec un contrôle d'accès basé sur les rôles (RBAC) s'appuyant sur le champ `role` du modèle utilisateur. Les **données sensibles** — documents KYC, pièces d'identité — sont stockées sur un service objet dédié (Cloudflare R2) avec accès restreint, et ne transitent par le service IA que le temps du traitement, sans persistance côté IA. Toutes les **entrées** des endpoints d'IA sont validées par des schémas Pydantic, ce qui rejette en amont les requêtes malformées. Le découplage des services limite par ailleurs la **surface d'attaque** : le service IA n'est pas exposé publiquement et n'est joignable que par le backend, au sein du réseau interne. Enfin, le caractère **entièrement local** des modèles d'IA (aucune API tierce) garantit qu'aucune donnée personnelle n'est transmise à un service externe, un atout majeur du point de vue de la confidentialité.

Conclusion

Ce chapitre a détaillé la conception de *The Communiun*, depuis l'architecture micro-services en trois tiers organisée en monorepo jusqu'à la formalisation de chaque algorithme du service d'Intelligence Artificielle, en passant par le modèle de données PostgreSQL/*pgvector* et la modélisation fonctionnelle UML. Les choix d'architecture — découplage NestJS/FastAPI, vecteurs intégrés à PostgreSQL, modèles d'IA locaux et chargés en singleton — répondent directement aux contraintes identifiées aux chapitres précédents : trilinguisme, absence de GPU, confidentialité et scalabilité. Le chapitre suivant présente la réalisation concrète de cette conception, les interfaces obtenues, ainsi que

la stratégie de tests et l'évaluation chiffrée des résultats.

Chapitre 4

Réalisation et Analyse des Résultats

Introduction

4.1 Environnement Technique

4.2 Workflow de Développement

4.3 Présentation des Interfaces

4.3.1 Onboarding et KYC / KYB

4.3.2 Fil d'Actualité et Réseau Social

4.3.3 Marketplace et Enchères

4.3.4 Espace Mentorat

4.3.5 Messagerie Instantanée

4.3.6 Tableau de Bord Administrateur

4.4 Démonstration des Workflows IA

4.4.1 Création d'Annonce Enrichie par l'IA

4.4.2 Vérification KYC Automatisée

4.5 Stratégie de Tests et Validation

4.5.1 Tests Unitaires IA (pytest)

4.5.2 Tests d'Intégration API

4.5.3 Validation des Données

4.6 Évaluation et Métriques

4.6.1 Métriques OCR

4.6.2 Métriques de Recommandation

4.6.3 Métriques de Détection (Anomalies, Fraude, Churn)

4.6.4 KPI Business et Impact

Conclusion

Conclusion Générale et Perspectives

Annexe A

Structure du Monorepo

Afin de faciliter la maintenance, le partage de code et le déploiement continu, le projet *The Commu-nium* est organisé sous forme de monorepo. Les captures d'écran ci-dessous, issues de l'environnement de développement, illustrent la séparation claire entre les applications principales et les paquets partagés.

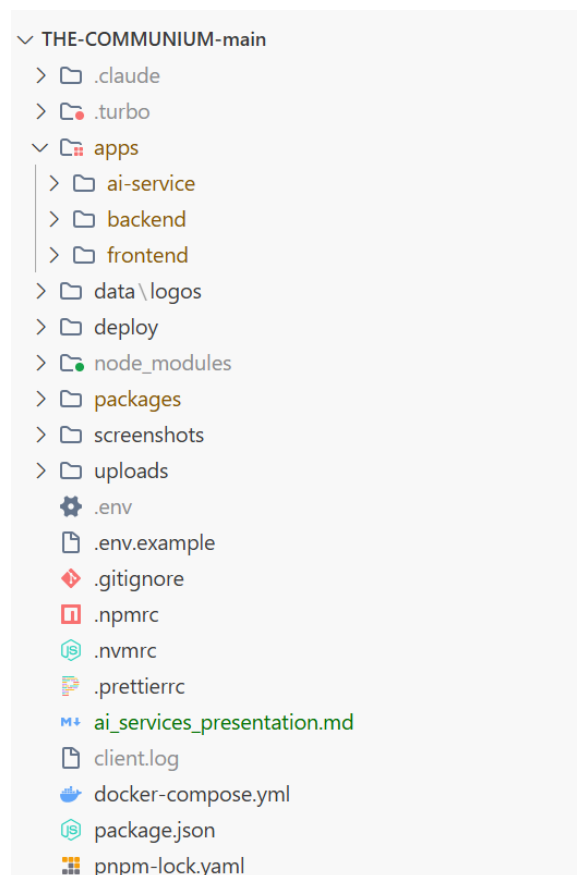


FIGURE A.1 – Architecture globale du monorepo (Turborepo/pnpm) séparant les applications (apps) et les modules partagés (packages).

Zoom sur le Microservice d'Intelligence Artificielle

Le backend IA, développé avec FastAPI, suit une architecture logicielle stricte séparant la définition des routes HTTP et la logique métier algorithmique.

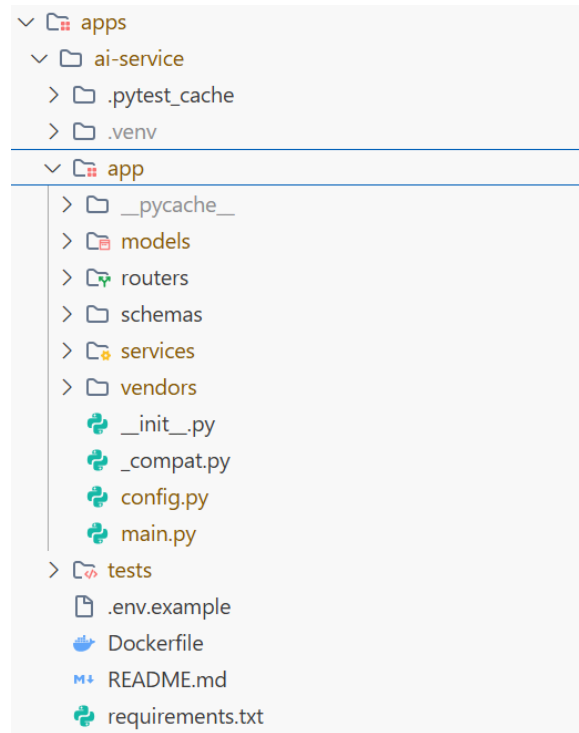


FIGURE A.2 – Arborescence principale de l'application ai-service, montrant la configuration Docker, les tests et le module central app.

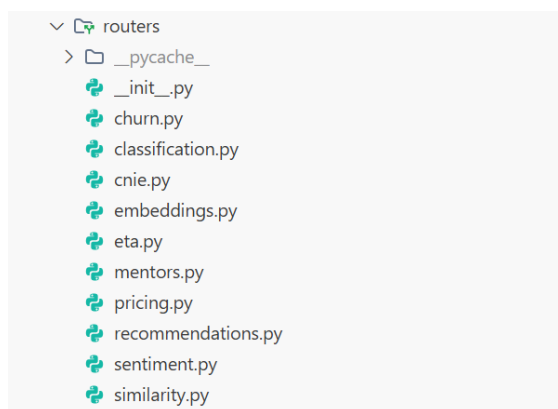


FIGURE A.3 – Détail du dossier routers définissant les points de terminaison de l'API.

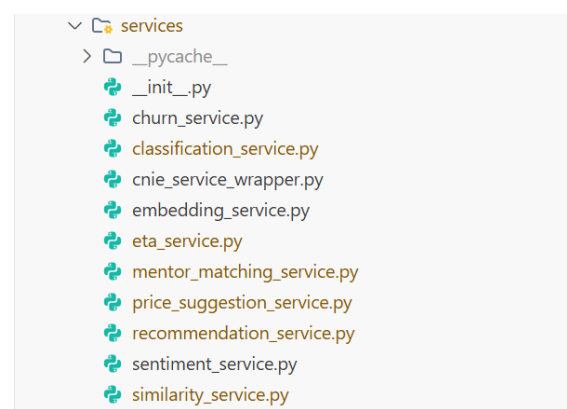


FIGURE A.4 – Détail du dossier services contenant les algorithmes d'IA et de Machine Learning.

Annexe B

Schéma Prisma (Extraits Clés)

Le paquet partagé database centralise l'accès aux données pour l'ensemble du monorepo. La figure ci-dessous illustre son arborescence, incluant le fichier de schéma, le dossier des migrations historiques et la configuration de l'ORM.

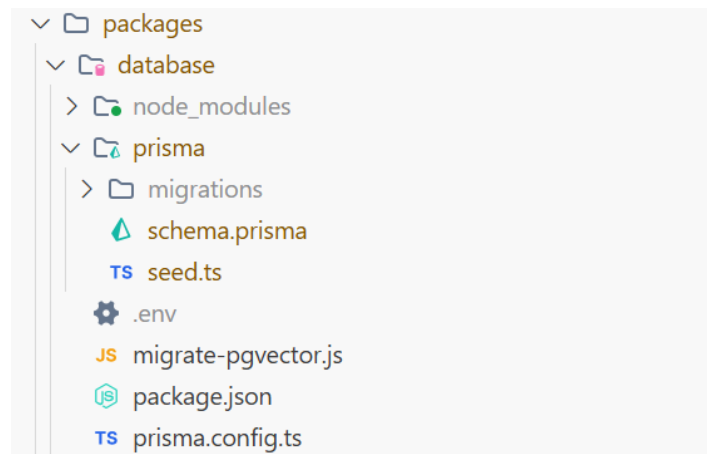


FIGURE B.1 – Structure du paquet partagé database, contenant le schéma Prisma, les migrations et le client ORM utilisé par l'ensemble du projet.

Le code suivant présente un extrait du fichier `schema.prisma` localisé dans ce répertoire. Il illustre la modélisation de la base de données PostgreSQL, mettant en évidence les relations entre le cœur du système (les utilisateurs), le module e-commerce (les annonces) et le module d'accompagnement (les profils de mentors).

```
// schema.prisma
```

```
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}
```

```

// -----
// Modèle Utilisateur Central
// -----
model User {
    id            String    @id @default(cuid())
    name          String?
    email         String?   @unique
    passwordHash  String?
    role          Role      @default(USER)
    createdAt     DateTime  @default(now())

    // Relations
    listings      Listing[]
    mentorProfile MentorProfile?
}

// -----
// Modèle pour la Marketplace (Annonces)
// -----
model Listing {
    id            String    @id @default(cuid())
    title         String
    description    String
    price         Float
    category      String
    condition     String
    status        Status    @default(ACTIVE)
    createdAt     DateTime  @default(now())

    // Relations
    sellerId      String
    seller        User      @relation(fields: [sellerId], references: [id])
}

```

```

// -----
// Modèle pour le module de Mentoring
// -----

model MentorProfile {
    id          String    @id @default(cuid())
    userId      String    @unique
    user        User      @relation(fields: [userId], references: [id])
    specialty   String
    yearsExp    Int
    rating       Float     @default(0.0)
    isAvailable Boolean    @default(true)
}

enum Role {
    USER
    ADMIN
    MENTOR
}

enum Status {
    ACTIVE
    SOLD
    DRAFT
}

```

Annexe C

Endpoints du Microservice IA

Cette annexe présente l'interface de documentation interactive (Swagger UI / OpenAPI 3.1) auto-générée par FastAPI. Elle offre une vue d'ensemble exhaustive des 10 points de terminaison (endpoints) développés pour encapsuler les modèles d'Intelligence Artificielle de la plateforme *The Communion*.

Vue globale de l'API

Communion AI Service 1.0.0 OAS 3.1
[/openapi.json](#)
Embedding and similarity service for The Communion

embeddings

POST `/embeddings/listing` Embed Listing

POST `/embeddings/mentor` Embed Mentor

similarity

POST `/similarity/listings` Find Similar Listings

FIGURE C.1 – En-tête de l'API et points de terminaison pour la génération d'embeddings multilingues et le calcul de similarité vectorielle.

mentors

POST `/mentors/match` Match Mentors

pricing

POST `/pricing/suggest` Suggest Listing Price

classification

POST `/classify/listing` Classify Listing Endpoint

sentiment

POST `/sentiment/analyze` Analyze

FIGURE C.2 – Points de terminaison dédiés à l'appariement hybride des mentors, la suggestion de prix, la classification NLP *Zero-Shot* et l'analyse de sentiment.

churn		^
POST	/churn/predict Predict Churn	✓
recommendations		^
POST	/recommendations/listings Get Listing Recommendations	✓
eta		^
POST	/eta/predict Predict Delivery Eta	✓
cnie		^
POST	/cnie/extract Extract Cnie	✓
POST	/cnie/extract-base64 Extract Cnie Base64	✓

FIGURE C.3 – Points de terminaison pour la prédiction d’attrition (Churn RFM), les recommandations diversifiées (MMR), l’estimation des délais (ETA) et l’extraction de données par vision par ordinateur (OCR CNIE).

default		^
GET	/health Health	✓
Schemas		^
Body_extract_cnie_cnie_extract_post > Expand all object		
CNIEExtractBase64Request > Expand all object		
CNIEProfileResponse > Expand all object		
CandidateEmbedding > Expand all object		
ChurnPredictionRequest > Expand all object		

FIGURE C.4 – Route de vérification de l’état du serveur (Health Check) et aperçu des schémas de données de validation générés via Pydantic.

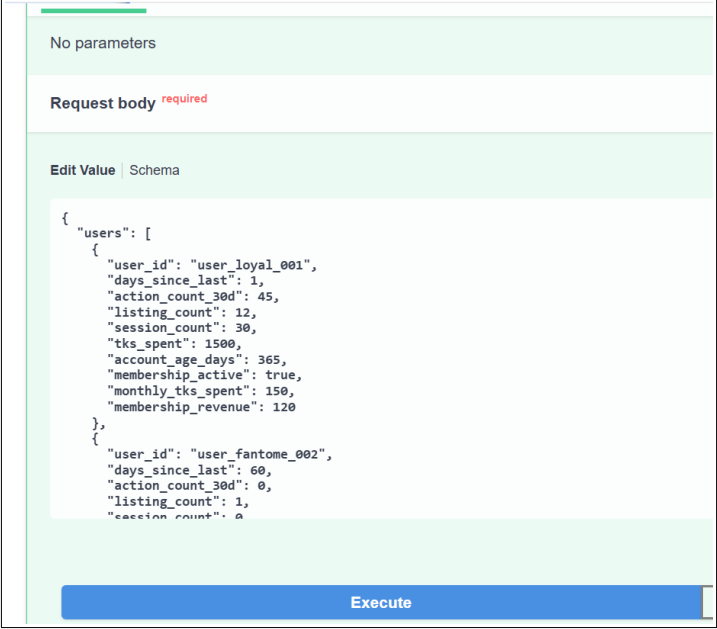
Annexe D

Résultats des Tests Unitaires

Cette annexe présente la validation fonctionnelle du microservice d'Intelligence Artificielle développé pour *The Communium*. Chaque modèle (NLP, vision par ordinateur, apprentissage automatique) a été testé de bout en bout via l'interface interactive Swagger UI. Pour chaque service, nous reproduisons la requête envoyée au serveur local et la réponse JSON renvoyée en temps réel.

D.1 Prédiction d'Attrition (Churn) et Réengagement

Le modèle d'analyse RFM détecte les utilisateurs à risque et propose des actions de réengagement personnalisées.



The screenshot shows a web interface with a light green header and a white main area. At the top, it says "No parameters". Below that, a section titled "Request body" with a red "required" label is shown. Underneath, there are two tabs: "Edit Value" (selected) and "Schema". The "Edit Value" tab displays a JSON object with two user entries. The first user, "user_loyal_001", has various activity metrics. The second user, "user_fantome_002", has fewer metrics. At the bottom, there is a blue "Execute" button.

```
{
  "users": [
    {
      "user_id": "user_loyal_001",
      "days_since_last": 1,
      "action_count_30d": 45,
      "listing_count": 12,
      "session_count": 30,
      "tks_spent": 1500,
      "account_age_days": 365,
      "membership_active": true,
      "monthly_tks_spent": 150,
      "membership_revenue": 120
    },
    {
      "user_id": "user_fantome_002",
      "days_since_last": 60,
      "action_count_30d": 0,
      "listing_count": 1,
      "session_count": 0
    }
  ]
}
```

FIGURE D.1 – Requête JSON contenant les données d'activité de deux utilisateurs distincts.



The screenshot shows a JSON object with two user entries. The first user, "user_loyal_001", has a "recommended_actions" array with three items: "send_re_engagement_email", "push_notification_new_content", and "suggest_forum_or_event". The second user, "user_qui_glisse_003", has a "recommended_actions" array with two items: "suggest_forum_or_event" and "highlight_new_listings". The JSON also includes "clv_estimate" and "clv_tier" for each user, as well as counts for high, medium, and low risk levels.

```
{
  "recommended_actions": [
    "send_re_engagement_email",
    "push_notification_new_content",
    "suggest_forum_or_event"
  ],
  "clv_estimate": 0,
  "clv_tier": "LOW"
},
{
  "user_id": "user_qui_glisse_003",
  "churn_score": 0.4545,
  "risk_level": "MEDIUM",
  "rfm_recency": 3,
  "rfm_frequency": 2,
  "rfm_monetary": 5,
  "recommended_actions": [
    "suggest_forum_or_event",
    "highlight_new_listings"
  ],
  "clv_estimate": 664.58,
  "clv_tier": "HIGH"
}
],
"high_risk_count": 1,
"medium_risk_count": 1,
"low_risk_count": 1
}
```

FIGURE D.2 – Génération dynamique d'actions de réengagement.

D.2 Extraction de Données via Vision par Ordinateur (OCR CNIE)

Ce test valide la lecture et l'extraction de données structurées à partir d'une image de carte d'identité marocaine (CNIE).

cnie

⌵

POST

/cnie/extract

Extract Cnie

⌵

Parameters

Cancel

Reset

No parameters

Request body required

multipart/form-data ⌵

file required

string(\$binary)

Choisir un fichier

CIN 1.jpeg

Execute

Clear

FIGURE D.3 – Interface de test Swagger pour le téléversement sécurisé (multipart/form-data) d’une image de CNIE.

	http://localhost:8000/cnie/extract
Server response	
Code	Details
200	Response body <pre>{ "card_number": "C0939432", "surname": "TALEB", "given_name": "SAMI", "full_name": "TALES SAMI", "birth_date": "23/11/2003", "birth_place": "NNAME lL QUARD FES", "address": null, "nationality": "Marocaine", "ocr_text": "ROYAUME DU MAROC\نجمية\NCARTE NATIONALE D'IDENTITE\فوقية\في شمس\SAAMI\شخصي\NTALEB\NMB 10\N23.11.2003\ازدادت\NNAME\N+l\QUARD\FES\N20-23\N\اشهد السيد\N\C0939432\nValidable jusquau 21 04 2031 إلى غاية", "confidence": 0 }</pre> Download Response headers <pre>content-length: 626 content-type: application/json date: Mon, 15 Jun 2026 14:30:35 GMT server: uvicorn</pre>
Responses	

FIGURE D.4 – Résultat de l'extraction OCR : identité complète, numéro de carte et date de naissance.

D.3 Classification Zero-Shot et Extraction de Tags (NLP)

Ces tests vérifient le nettoyage des mots vides (*stopwords*) et la pertinence de la catégorisation automatique des annonces.

Predict delivery / fulfilment ETA for a marketplace listing.

Inputs are the features a seller/platform has at listing-creation time: category, city, condition, seller activity h

The model applies a feature-engineered regression (city tier × category speed × condition × seller experien

Returns:

- eta_minutes — canonical prediction in minutes
- eta_hours — human-readable float
- eta_days — celled days (for UI badges)
- confidence — model confidence [0–1]
- breakdown — per-factor explanation

Parameters

No parameters

Request body required

Edit Value | Schema

```
{
  "category_name": "Matériel Informatique",
  "seller_city": "Rabat",
  "buyer_city": "Rabat",
  "condition": "new",
  "seller_listing_count": 120,
  "hour_of_day": 14,
  "day_of_week": 1
}
```

FIGURE D.5 – Soumission d’une annonce brute pour analyse sémantique.

Request URL

http://localhost:8000/classify/listing

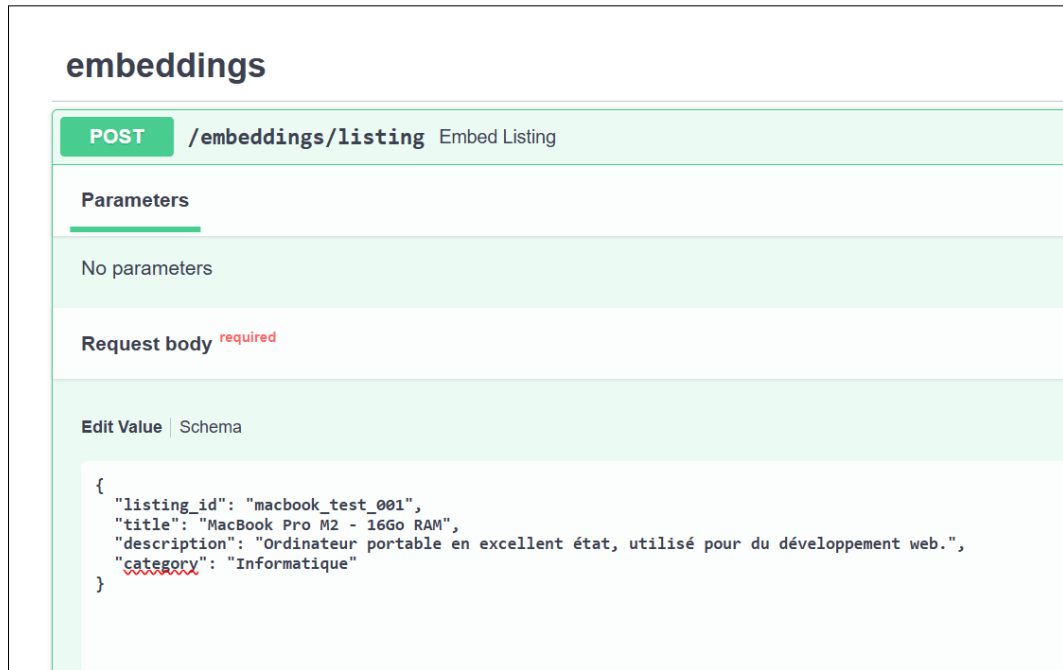
Server response

Code	Details
200	Response body <pre>{ "predicted_category": "Informatique & Tech", "confidence": 0.8026, "suggested_tags": ["développeur", "react", "startup", "important"], "urgency": "HIGH" }</pre> Response headers

FIGURE D.6 – Réponse du service : catégorie technique, niveau d’urgence et liste des tags pertinents.

D.4 Moteur de Recherche Sémantique : Génération d'Embeddings

La vectorisation est le cœur du système de similarité. Ce test confirme la transformation d'un texte en coordonnées mathématiques exploitables.



embeddings

POST /embeddings/listing Embed Listing

Parameters

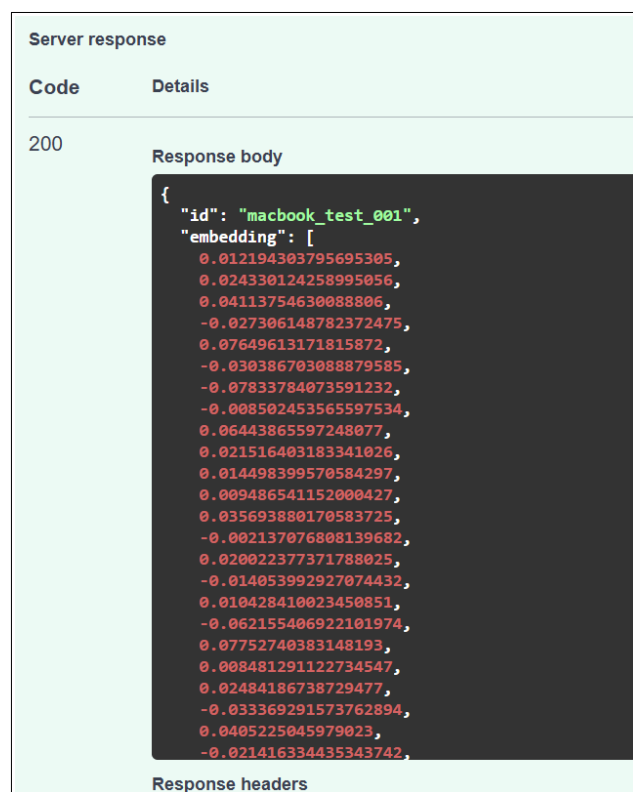
No parameters

Request body required

[Edit Value](#) | [Schema](#)

```
{
  "listing_id": "macbook_test_001",
  "title": "MacBook Pro M2 - 16Go RAM",
  "description": "Ordinateur portable en excellent état, utilisé pour du développement web.",
  "category": "Informatique"
}
```

FIGURE D.7 – Requête envoyant le titre et la description d'une annonce (ex. : MacBook Pro).



Server response

Code	Details
200	Response body <pre>{ "id": "macbook_test_001", "embedding": [0.012194303795695305, 0.024330124258995056, 0.04113754630088806, -0.027306148782372475, 0.07649613171815872, -0.030386703088879585, -0.07833784073591232, -0.008502453565597534, 0.06443865597248077, 0.021516403183341026, 0.014498399570584297, 0.009486541152000427, 0.035693880170583725, -0.002137076808139682, 0.020022377371788025, -0.014053992927074432, 0.010428410023450851, -0.062155406922101974, 0.07752740383148193, 0.008481291122734547, 0.02484186738729477, -0.033369291573762894, 0.0405225045979023, -0.021416334435343742] }</pre> Response headers

FIGURE D.8 – Vecteur multidimensionnel (embedding) généré avec succès par le modèle NLP.

D.5 Prédiction des Délais de Livraison (Modèle ETA)

Ce test évalue l’algorithme d’estimation de délai fondé sur la distance géographique (*feature engineering*) et les conditions d’expédition du vendeur.

Predict delivery / fulfilment ETA for a marketplace listing.

Inputs are the features a seller/platform has at listing-creation time: category, city, condition, seller activity h

The model applies a feature-engineered regression (city tier × category speed × condition × seller experien

Returns:

- eta_minutes — canonical prediction in minutes
- eta_hours — human-readable float
- eta_days — celled days (for UI badges)
- confidence — model confidence [0–1]
- breakdown — per-factor explanation

Parameters

No parameters

Request body required

Edit Value | Schema

```
{  "category_name": "Matériel Informatique",  "seller_city": "Rabat",  "buyer_city": "Rabat",  "condition": "new",  "seller_listing_count": 120,  "hour_of_day": 14,  "day_of_week": 1}
```

FIGURE D.9 – Exemple de requête paramétrée pour le calcul du délai d’expédition.

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/eta/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "category_name": "Matériel Informatique",
    "seller_city": "Rabat",
    "buyer_city": "Rabat",
    "condition": "new",
    "seller_listing_count": 120,
    "hour_of_day": 14,
    "day_of_week": 1
  }'
```

FIGURE D.10 – Estimation du délai, prenant en compte la distance géographique (trajet interurbain).

D.6 Appariement Hybride des Mentors

Le service de *Mentor Matching* calcule un score hybride pondérant la pertinence sémantique, la note globale et l'expérience du mentor, avec une pénalité en cas d'indisponibilité.

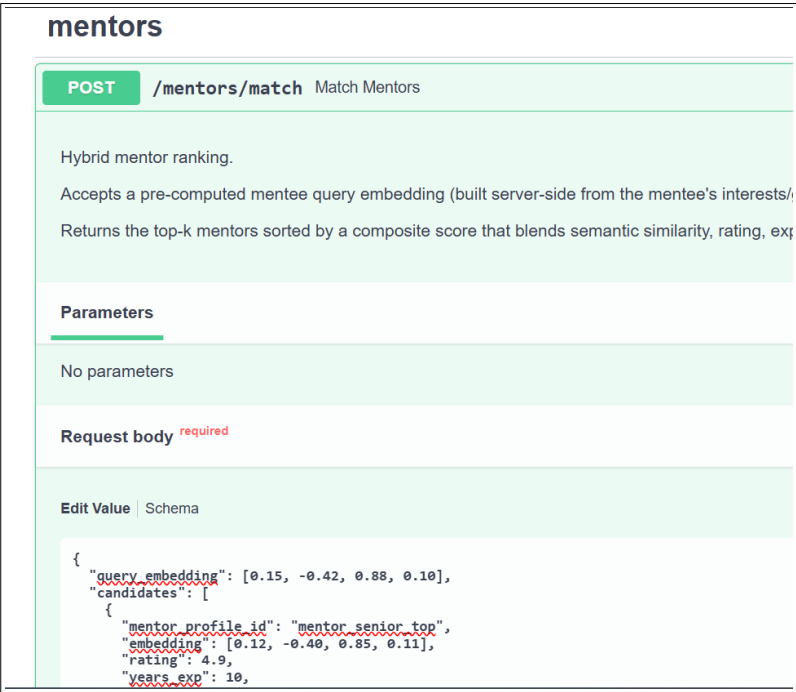


FIGURE D.11 – Requête envoyant le vecteur d'intérêts de l'utilisateur et les données des mentors candidats.

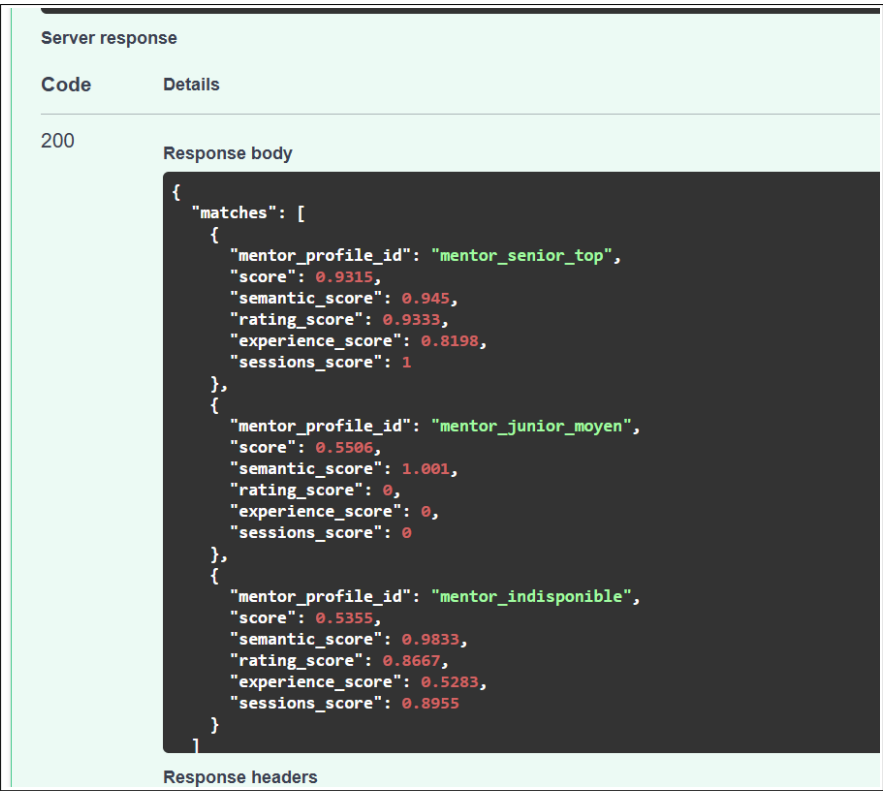


FIGURE D.12 – Classement final appliquant les pondérations métier et les filtres de disponibilité.

D.7 Algorithme de Recommandations Diversifiées (MMR)

Pour éviter l'effet de « chambre d'écho », l'algorithme de recommandation intègre la méthode *Maximal Marginal Relevance* (MMR).

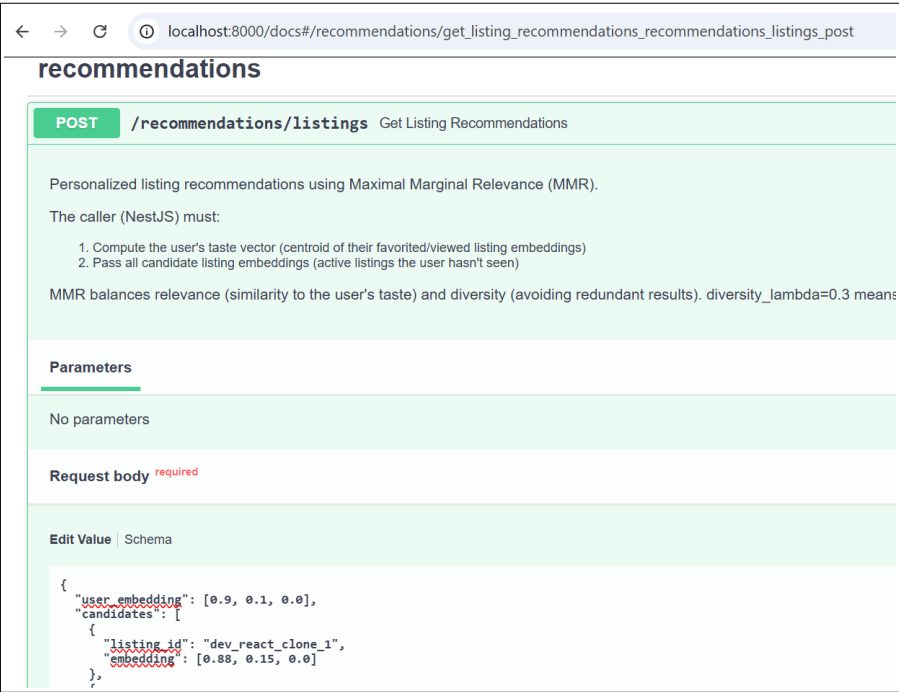


FIGURE D.13 – Configuration du point de terminaison, incluant le paramètre de diversité lambda.

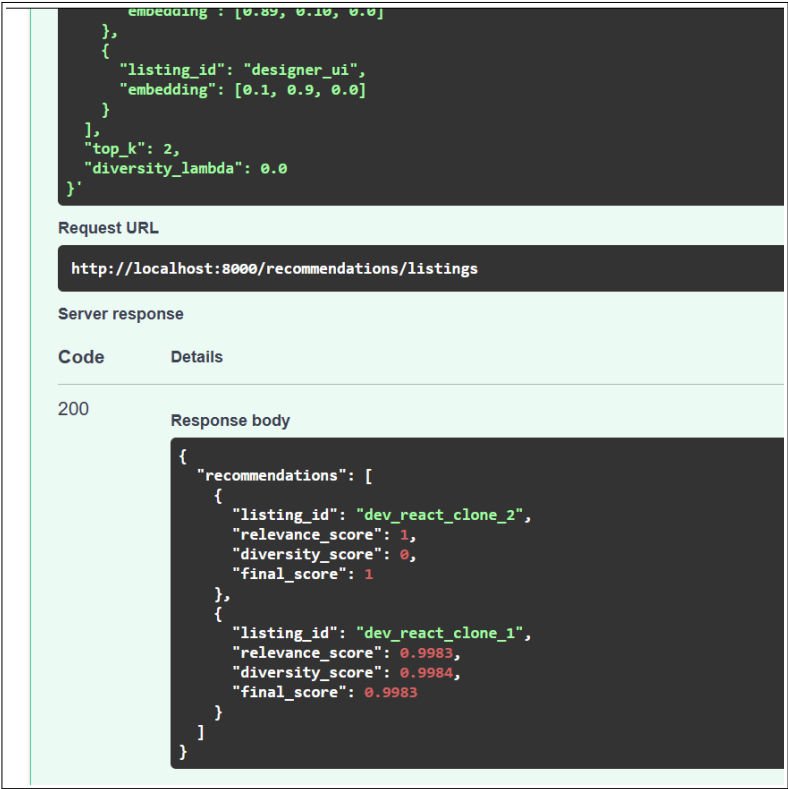


FIGURE D.14 – Résultat MMR : tri final équilibrant le score de pertinence et la pénalité de redondance.

D.8 Modèle d'Estimation de Prix (Pricing)

L'algorithme analyse les caractéristiques du produit (âge, état, valeur à neuf) pour proposer une estimation juste, même en situation de démarrage à froid (*Cold Start*).

pricing

POST `/pricing/suggest` Suggest Listing Price

Suggest a fair price range for a marketplace listing.

Provide `comparable_prices` (recent sold/active prices for the same category) for a data-driven estimate. If not provided, the heuristic returns a low confidence score so the caller can inform the user accordingly.

Parameters

No parameters

Request body *required*

[Edit Value](#) | [Schema](#)

```
{
  "title": "MacBook Pro M2 - 16Go RAM",
  "description": "Ordinateur portable en excellent état, très peu utilisé.",
  "category": "Matériel Informatique",
  "brand": "Apple",
  "condition": "used_good",
  "age_months": 24,
  "original_price": 15000
}
```

FIGURE D.15 – Requête soumettant les caractéristiques d'un ordinateur portable d'occasion (24 mois, bon état).

```
{
  "title": "MacBook Pro M2 - 16Go RAM",
  "description": "Ordinateur portable en excellent état, très peu utilisé.",
  "category": "Matériel Informatique",
  "brand": "Apple",
  "condition": "used_good",
  "age_months": 24,
  "original_price": 15000
}
```

Request URL

`http://localhost:8000/pricing/suggest`

Server response

Code	Details
200	Response body <pre>{ "min_price": 1, "max_price": 1, "recommended_price": 1, "confidence": 0.1, "method": "heuristic_cold" }</pre> Response headers <pre>content-length: 100 content-type: application/json date: Tue, 16 Jun 2026 14:55:43 GMT server: uvicorn</pre>

Responses

FIGURE D.16 – Déclenchement de la méthode heuristique de secours (*heuristic_cold*) calculant la décote du matériel.

D.9 Analyse de Sentiments Multilingue

Ce service évalue par lots (*batch*) la polarité de textes en français, anglais et arabe, sans téléchargement de modèles supplémentaires.

sentiment

POST `/sentiment/analyze` Analyze

Batch sentiment analysis for review texts.

Accepts up to 100 texts per request. Returns per-text label (POSITIVE / NEGATIVE / NEUTRAL), confidence score, and average compound for the whole batch.

Works on French, English, and Arabic without any extra model downloads.

Parameters

No parameters

Request body required

[Edit Value](#) | [Schema](#)

```
{  "texts": [    "L'interface de cette nouvelle plateforme est incroyablement fluide et le mode sombre est magnifique.",    "The application keeps crashing every time I try to update my profile. Very frustrating!",    "الخدمة جيدة بشكل عام، لكن التسجيل استغرق وقتًا طويلاً",    "Je n'ai pas d'avis particulier, c'est un outil standard."  ]}
```

FIGURE D.17 – Requête soumettant un lot de textes hétérogènes (anglais, français, arabe) pour analyse de polarité.

CodeDetails

200Response body

```
{  "results": [    {      "text": "L'interface de cette nouvelle plateforme est incroyablement fluide et le mode sombre est magnifique.",      "label": "POSITIVE",      "score": 0.5746,      "compound": 0.1491    },    {      "text": "The application keeps crashing every time I try to update my profile. Very frustrating!",      "label": "NEGATIVE",      "score": 0.6094,      "compound": -0.2187    },    {      "text": "الخدمة جيدة بشكل عام، لكن التسجيل استغرق وقتًا طويلاً",      "label": "NEGATIVE",      "score": 0.5587,      "compound": -0.1174    },    {      "text": "Je n'ai pas d'avis particulier, c'est un outil standard.",      "label": "POSITIVE",      "score": 0.5655,      "compound": 0.1311    }  ]}
```

FIGURE D.18 – Prédictions du modèle NLP : polarité (POSITIVE/NEGATIVE) et score de confiance par langue.

D.10 Recherche de Similarité Vectorielle

Cette brique compare le vecteur d'une requête cible à une base de candidats pour identifier les correspondances les plus proches, indépendamment des mots-clés exacts.

similarity

POST /similarity/listings Find Similar Listings

Parameters

No parameters

Request body required

Edit Value | **Schema**

```
{
  "listing_id": "item_query_123",
  "query_embedding": [0.12, 0.45, -0.99, 0.34],
  "candidate_embeddings": [
    {
      "listing_id": "cand_001",
      "embedding": [0.11, 0.42, -0.90, 0.31]
    },
    {
      "listing_id": "cand_002",
      "embedding": [-0.55, 0.88, 0.12, -0.05]
    },
    {
      "listing_id": "cand_003",
      "embedding": [0.15, 0.40, -0.85, 0.30]
    }
  ],
  "top_k": 2
}
```

FIGURE D.19 – Requête soumettant l’embedding d’une annonce cible et ceux de trois annonces candidates.

Request URL

http://localhost:8000/similarity/listings

Server response

Code	Details
200	Response body <pre>{ "listing_id": "item_query_123", "similar": [{ "id": "cand_001", "score": 1.1986 }, { "id": "cand_003", "score": 1.1415 }] }</pre> Response headers

FIGURE D.20 – Similarité cosinus : les deux annonces les plus pertinentes (top_k=2), triées par proximité.

Annexe E

Captures d'Écran de l'Application

Cette annexe présente l'ensemble des interfaces utilisateur de la plateforme *The Communium*, illustrant le parcours complet d'un utilisateur, depuis son inscription jusqu'à l'utilisation des services avancés (Marketplace, Forum, etc.).

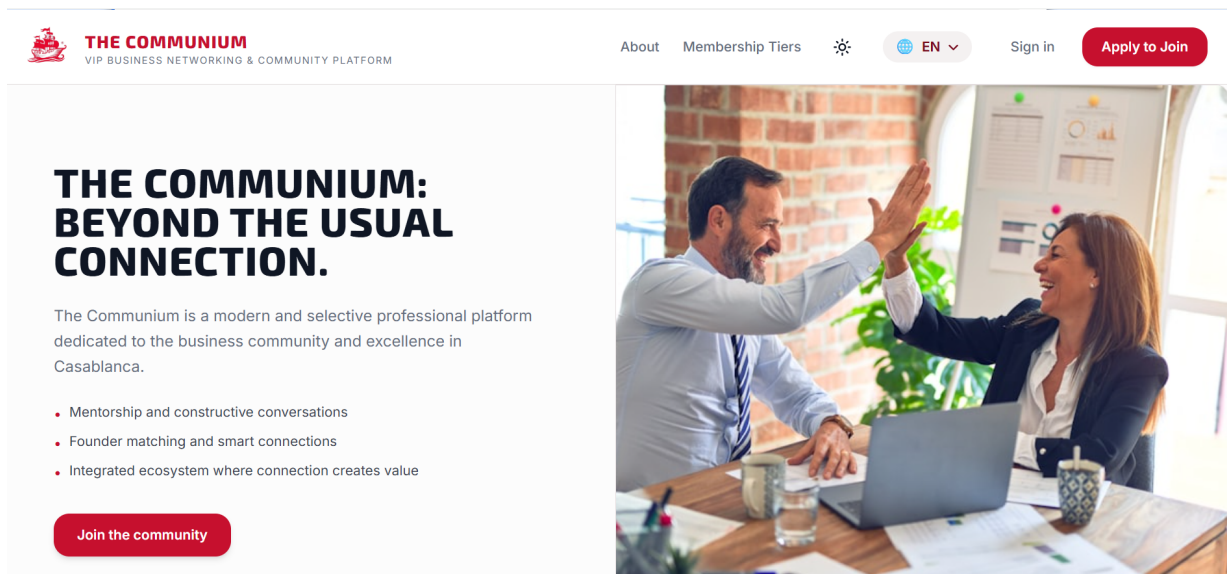
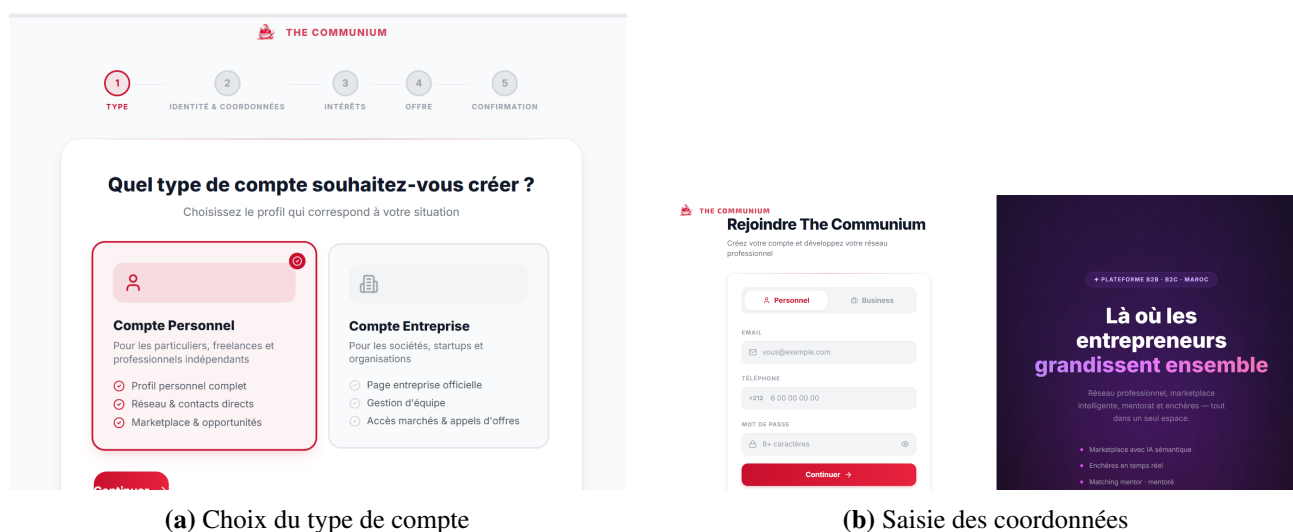


FIGURE E.1 – Page d'accueil (Landing Page) de The Communium



(a) Choix du type de compte

(b) Saisie des coordonnées

FIGURE E.2 – Processus d'inscription et de création de compte

Votre identité & Coordonnées

Renseignez vos informations d'identité et de contact (extraites automatiquement depuis votre CIN)

REPLISSAGE AUTOMATIQUE PAR IA (RECOMMANDÉ)

Uploadez le recto de votre CIN pour pré-remplir tous les champs automatiquement.

CHOISIR RECTO CIN (IMAGE)

PRÉNOM * User

NOM * Nom de famille

DATE DE NAISSANCE * jj/mm/aaaa

TYPE DE PIÈCE CIN

NUMÉRO * # BE 123456

(a) Étape 1 : Upload de la CIN

Champs préremplis avec succès !

PRÉNOM * SAMI

NOM * TALEB

DATE DE NAISSANCE * 23/11/2003

TYPE DE PIÈCE CIN

NUMÉRO * # C0939432

COORDONNÉES DE CONTACT

TÉLÉPHONE * 0713368237

VILLE * JNANE EL OUARD FES

PAYS * Marocaine

(b) Étape 2 : Extraction et auto-remplissage par IA

FIGURE E.3 – Processus de vérification d'identité (KYC)

Choisissez votre offre

Accédez à des privilèges exclusifs dès aujourd'hui

Gratuit

0 DHS/AN

- Profil complet
- Réseau de contacts
- Marketplace standard

✓ Sélectionné

Premium

4 \$/MOIS

- Badge VIP
- Réductions jusqu'à 90%
- Gagnez des TKS!
- Priorité support

Choisir ce plan

< Précédent

Continuer →

(a) Sélection de l'abonnement

Finalisez votre profil

Dernière étape — vérifiez vos informations et ajoutez votre photo

RÉCAPITULATIF

Type	Compte Personnel
Nom complet	SAMI TALEB
Pièce d'identité	C0939432
Ville	JNANE EL OUARD FES, Marocaine
Téléphone	0713368237
Intérêts	3 sélectionnés

Photo enregistrée

Une photo professionnelle augmente votre visibilité de 400% — vous pouvez l'ajouter plus tard.

(b) Ajout de la photo de profil

FIGURE E.4 – Finalisation de l'intégration (Onboarding)

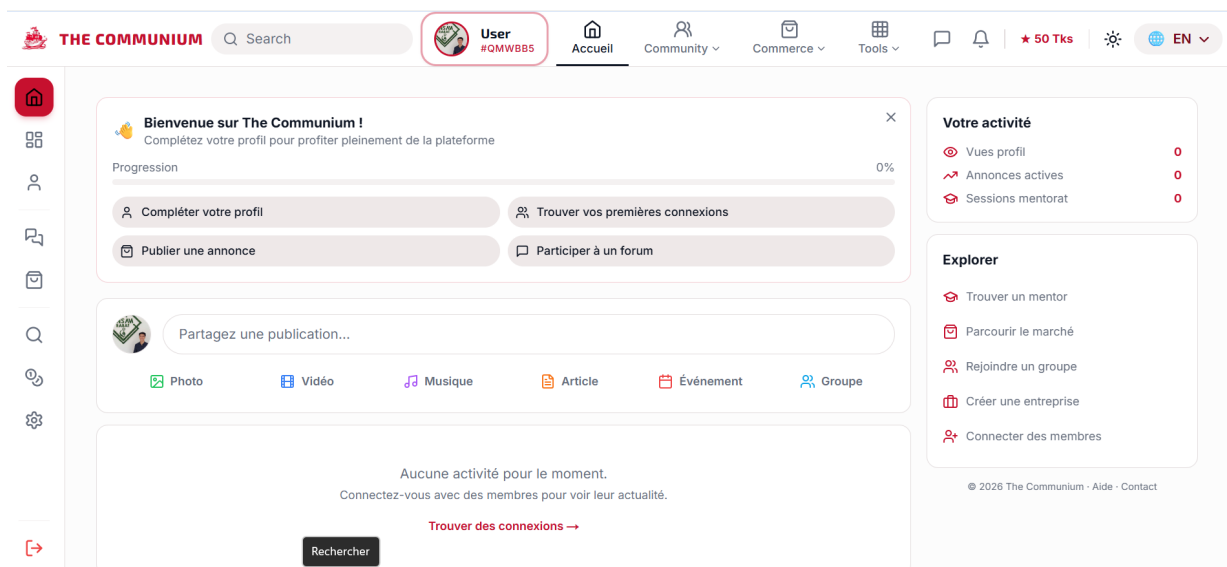


FIGURE E.5 – Fil d’actualité et suggestions de connexions

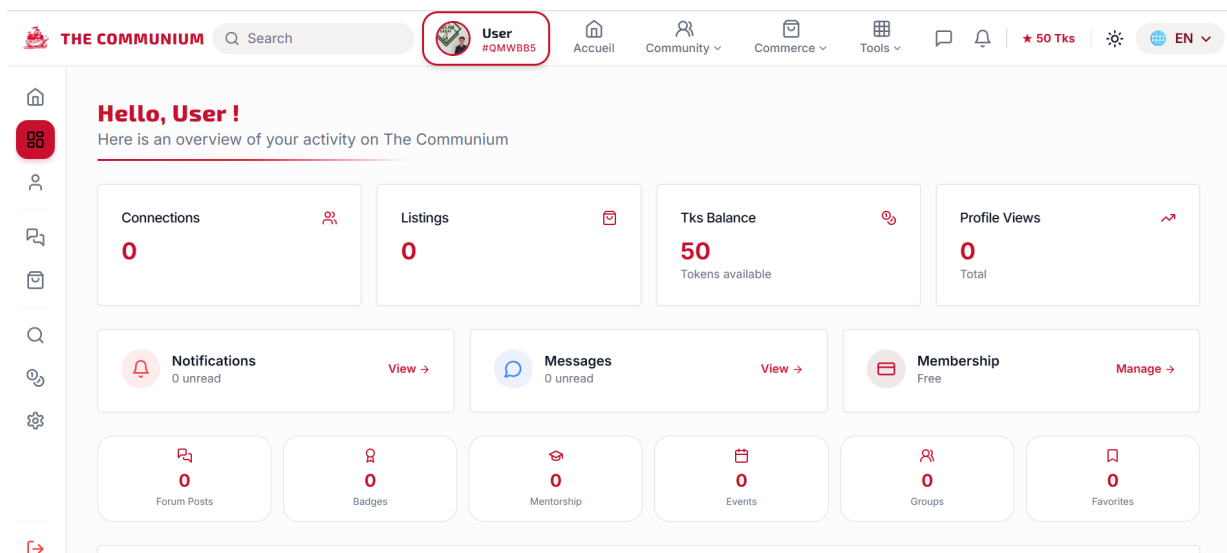


FIGURE E.6 – Tableau de bord (Dashboard) résumant l’activité de l’utilisateur

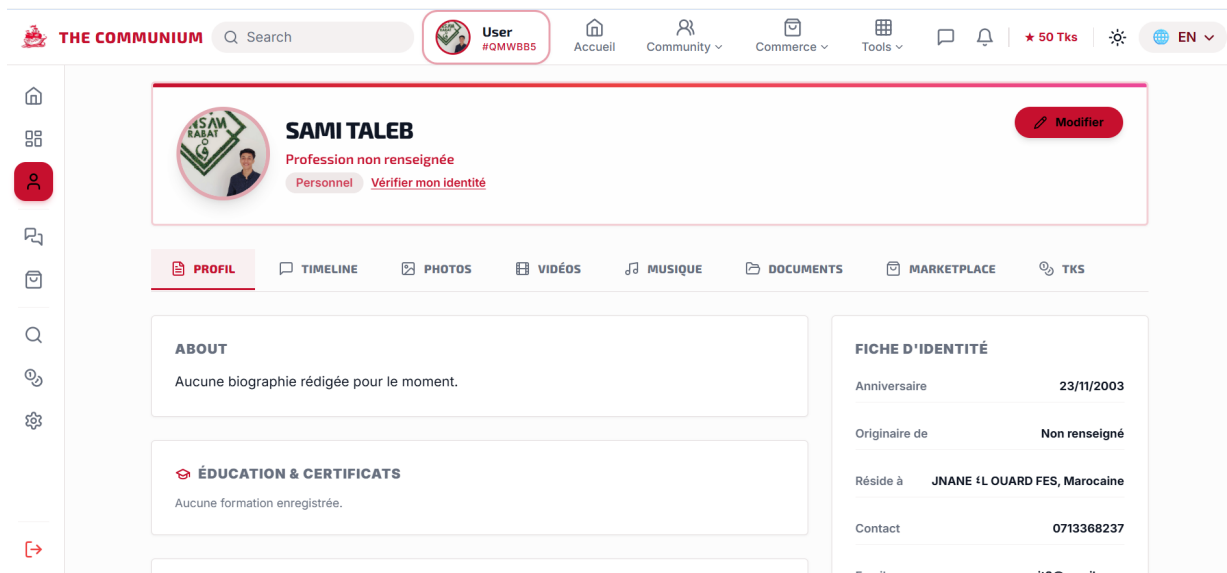


FIGURE E.7 – Vue détaillée du profil utilisateur et de sa fiche d'identité vérifiée

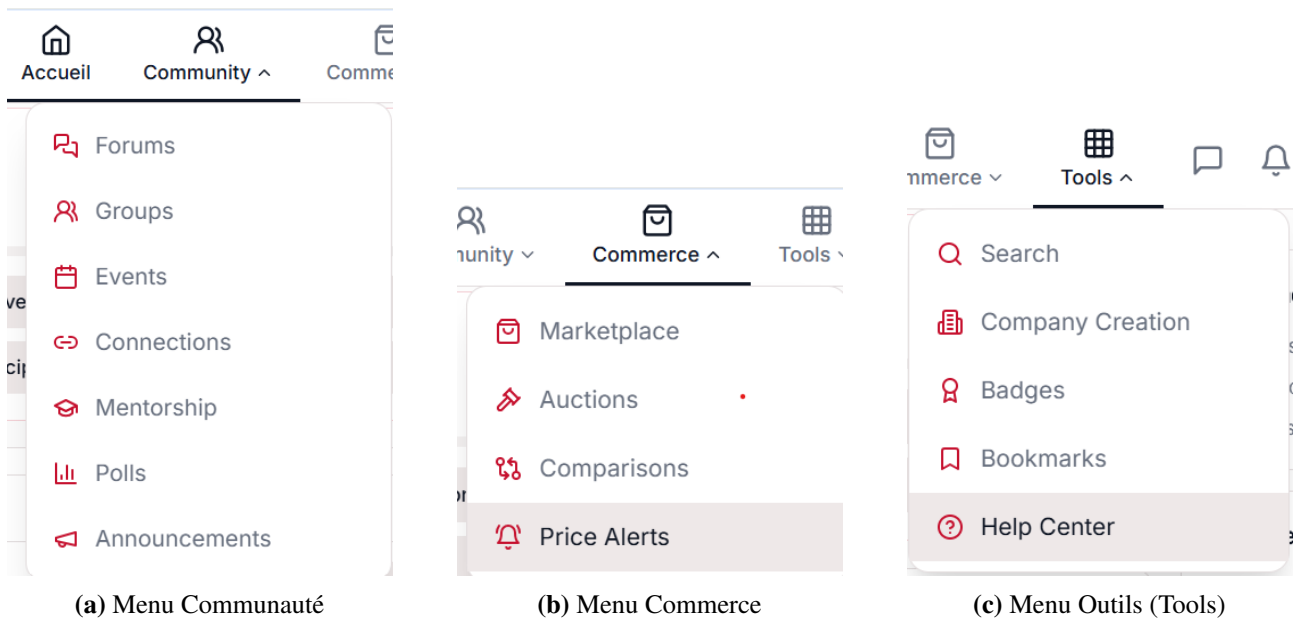


FIGURE E.8 – Menus de navigation principaux de la plateforme



FIGURE E.9 – Interface de l’espace Forum catégorisé

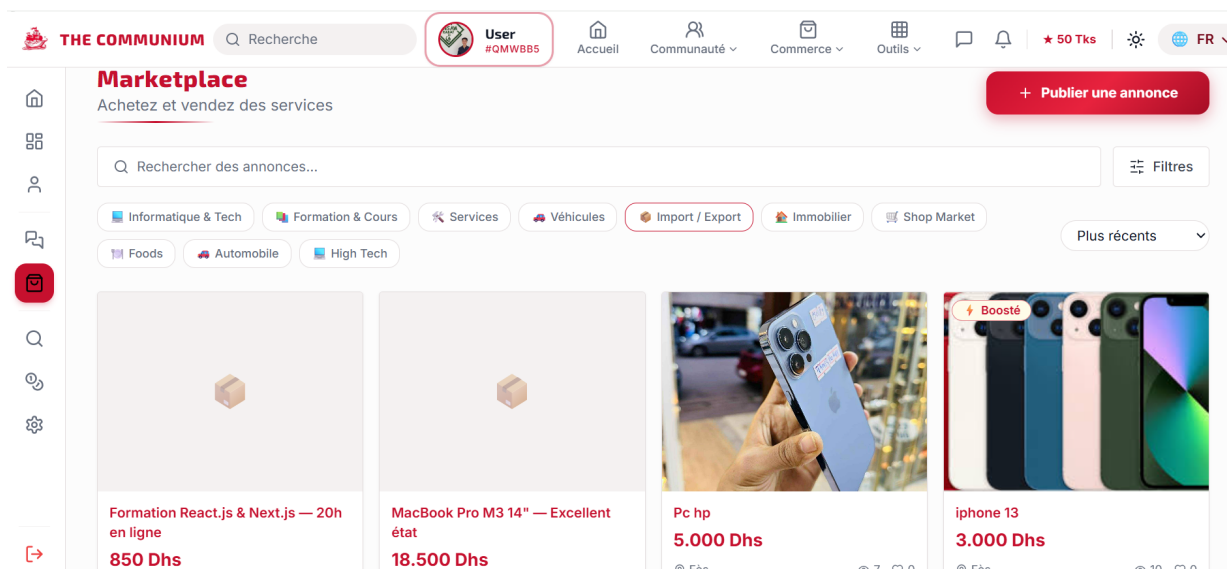


FIGURE E.10 – Marketplace B2B/B2C pour l’achat et la vente de services ou produits

 THE COMMUNIUM

Recherche

 User
#QMWB5

Accueil

Communauté

Commerce

Outils

50 Tks

FR











Paramètres

 **Mon compte**
Email, téléphone, langue

 **Connexion & sécurité**
Mot de passe, sessions

 **Notifications**
Email, push, in-app

 **Visibilité du profil**
Qui peut voir vos infos

 **Abonnement**
Plan, facturation

 **Confidentialité & donn...**
Données, suppression

 **Mon compte**
Gérez vos informations de compte

 **Adresse email**
samit2@gmail.com

 **Téléphone**
0713368237

 **Langue**
Français

 **Type de compte**
Personnel

Modifier

Modifier

Modifier

Pour modifier votre nom et photo, visitez [la page profil](#).

FIGURE E.11 – Page de gestion des paramètres et de la sécurité du compte